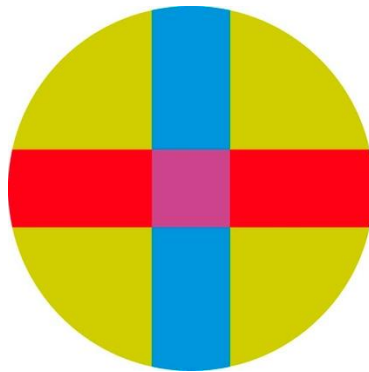


UNIVERSIDAD SAN PABLO - CEU

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE INFORMACIÓN



TRABAJO FIN DE GRADO

Recomendación nutricional a partir de imágenes

Nutritional Recommendation Based on Images

Autor: Javier Martínez Cristóbal

Tutor: Raúl A. del Águila Escobar

Raúl García García

Julio 2025



UNIVERSIDAD SAN PABLO-CEU

ESCUELA POLITÉCNICA SUPERIOR

División de Ingeniería

Calificación del Trabajo Fin de Grado

Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/_____, acuerda otorgar al Trabajo Fin de Grado presentado por D./Dña. _____ la calificación de _____

Agradecimientos

En primer lugar, quisiera expresar mi más profundo agradecimiento a mi familia. Vuestro apoyo incondicional, paciencia y ánimo han sido el pilar fundamental durante todos estos años de carrera. Gracias por creer en mí, por vuestro sacrificio y por cada palabra que me ha impulsado a seguir adelante y a superar cada obstáculo en el camino hasta llegar a este momento tan importante.

También, extiendo mi gratitud a mis tutores de este Trabajo de Fin de Grado, por su guía, su constante disponibilidad y orientación. Gracias por la paciencia y por motivarme a dar lo mejor de mí.

Por último, deseo agradecer a la Universidad San Pablo CEU por brindarme la oportunidad de formarme profesionalmente. A todos los profesores que han sido parte de mi educación, gracias por transmitir su conocimiento y pasión con tanta dedicación. A mis compañeros y amigos, gracias por las experiencias compartidas, el apoyo mutuo y aprendizaje que han enriquecido enormemente estos años universitarios.

Resumen

Este trabajo de fin de grado consiste en el desarrollo de un sistema de recomendación nutricional que combina visión por computador, grafos de conocimiento y computación sin servidores. El usuario interactúa mediante un *bot* de Telegram que, tras un breve cuestionario sobre preferencias (calorías, grasas, azúcar, etc.), permite introducir ingredientes manualmente o enviando una foto. Si se aporta una imagen, un modelo YOLOv8 detecta los ingredientes presentes; en ambos casos se publica la lista de ingredientes en un *topic* Pub/Sub.

Una función “Recomendador” en Google Cloud recibe los ingredientes y las preferencias, consulta un grafo de conocimiento embebido con PyKEEN y calcula las recetas más adecuadas sumando la puntuación relacional de múltiples nutrientes. La respuesta formateada viaja a otro tópico y se envía al usuario a través de una función “Mensajero”. Toda la tubería está desplegada con Cloud Functions Gen2 y escala bajo demanda, minimizando costes y complejidad operativa mientras mantiene la modularidad necesaria para futuras mejoras.

Las evaluaciones preliminares muestran un mAP del 82 % en detección y hits@10 del 99 % en recomendación, lo que confirma la viabilidad del enfoque.

Palabras Clave

Transfer Learning, YOLOv8, Detección de Objetos, Bot de Telegram, Sistemas de Recomendación, Knowledge Graph Embeddings, PyKeen, Google Cloud Platform, Cloud Functions, Pub/Sub, Nutrición, Serverless.

Abstract

This Final Degree Project focuses on developing a nutritional recommendation system that merges computer vision, knowledge graphs, and serverless computing. Users interact through a Telegram bot that, after a brief questionnaire about their preferences (calories, fats, sugar, etc.), lets them enter ingredients manually or by uploading a photo. When an image is provided, a YOLOv8 model detects the ingredients; in either case, the resulting ingredient list is published to a Pub/Sub topic.

A “Recommender” Cloud Function on Google Cloud receives both the ingredients and user preferences, queries a PyKEEN-embedded knowledge graph, and ranks the most suitable recipes by summing relational scores across multiple nutrients. The formatted response is sent to another topic and finally delivered to the user via a “Messenger” function. The entire pipeline is deployed with Cloud Functions Gen2, scaling on demand to minimize costs and operational complexity while preserving the modularity required for future enhancements.

Preliminary evaluations report an 82 % mAP for detection and a 99 % hits@10 for recommendation, confirming the feasibility of the approach.

Keywords

Transfer Learning, YOLOv8, Object Detection, Telegram Bot, Recommender Systems, Knowledge Graph Embeddings, PyKEEN, Google Cloud Platform, Cloud Functions, Pub/Sub, Nutrition, Serverless.

Índice de contenidos

1. Introducción.....	8
1.1. Contexto del TFG	8
1.2. Solución propuesta	9
1.3. Objetivos	10
2. Gestión del proyecto	11
2.1. Modelo de ciclo de vida	11
2.2. Planificación	11
2.4. Recursos necesarios	15
2.5. Justificación tecnológica	18
3. Análisis	22
3.1. Especificación de requisitos	22
3.2. Casos de Uso	23
3.3. Definición de las interacciones y su secuencia en el sistema	25
4. Arquitectura del sistema	26
4.1 Contexto	26
4.2 Preparación del recomendador basado en KGE	28
4.3 Preparación del modelo de reconocimiento de imágenes	32
4.4 Arquitectura del sistema	37
4.4.1. Arquitectura lógica	38
4.4.2. Arquitectura física	41
4.5 Bot	43
4.6 Seguridad	44
4.7 Referencia al repositorio de software.....	45
5. Validación del sistema	46
5.1. Plan de pruebas	46
5.2. Evaluación del sistema	47
6. Conclusiones y líneas futuras	51
6.1. Conclusiones	51

6.2. Líneas futuras	52
Bibliografía.....	53
Anexo I – Análisis de la distribución de nutrientes	55
Anexo II – Resultados de Optimización de Hiperparámetros (HPO)	64
Glosario de términos	66

Índice de figuras

Figura 1. Diagrama de Gantt Inicial	13
Figura 2. Diagrama de Gantt Real.....	14
Figura 3. Google Cloud Platform costes actuales	17
Figura 4. Casos de uso	23
Figura 5. Diagrama de secuencia del sistema	25
Figura 6. Preparación dataset KGE	28
Figura 7. Pasos seguidos para la generación del recomendador basado en KGE	30
Figura 8. Despliegue de modelo KGE en Google Cloud	32
Figura 9. Preparación del dataset y selección de modelo YOLOv8	33
Figura 10. Pasos seguidos para el modelo de detección basado en YOLO	34
Figura 11. Despliegue del modelo de detección de imágenes en Google Cloud	36
Figura 12. Componentes arquitectura lógica del sistema.....	38
Figura 13. Flujo arquitectura lógica.....	40
Figura 14. Arquitectura física del sistema	41

Índice de tablas

Tabla 1. Calendarización bloques de tareas	12
Tabla 2. Recursos software.....	15
Tabla 3. Recursos Google Cloud Platform	16
Tabla 4. Requisitos.....	22
Tabla 5. Resultados obtenidos del entrenamiento del modelo basado en KGE	31

1.Introducción

1.1. Contexto del TFG

En nuestro día a día, la decisión sobre qué cocinar a partir de los ingredientes disponibles en casa puede ser un desafío. Este problema no solo lleva a una falta de variedad en la dieta, sino que también es una de las principales causas del desperdicio de alimentos. Diversos estudios señalan que la falta de tiempo y el desconocimiento culinario son barreras habituales para seguir una dieta variada y saludable. De hecho, una encuesta de 2019 de Nutrisystem que se realizó a 2.000 adultos reveló que el 55% ni prueba recetas nuevas porque considera que tardan demasiado en prepararse, mientras que un 45% admite no saber cómo combinar ingredientes de forma saludable (Melore, 2021). En la misma línea, un informe de la consultora Spoon Guru indica que el 76% de los consumidores “necesita ayuda para identificar opciones saludables” en su día a día, pese a manifestar interés por mejorar su alimentación (Team, 2019).

El presente Trabajo de Fin de Grado (TFG) aborda este problema mediante el diseño y la implementación de un sistema inteligente de recomendación de recetas. El objetivo principal es ofrecer al usuario una solución rápida, interactiva y personalizada que responda a la pregunta: "¿Qué puedo cocinar con lo que tengo?". Para ello, el sistema integra técnicas de visión por computador y procesamiento del lenguaje natural, orquestadas en una arquitectura de *software* moderna y escalable.

1.2. Solución propuesta

La solución desarrollada consiste en un asistente de cocina virtual, accesible a través de un *bot* de Telegram que el usuario preconfigura según sus preferencias y necesidades nutricionales.

El funcionamiento se resume así:

1. **El usuario charla con el *bot*.** Primero responde un breve cuestionario sobre sus preferencias (calorías, grasas, etc.). Después elige entre escribir los ingredientes o enviar una foto de ellos.
2. **Si hay foto, se reconocen los ingredientes.** El sistema pasa la imagen a un modelo YOLOv8, sobre el que se ha realizado *fine tuning*, que identifica cada alimento que aparece en la foto. Si el usuario ya los escribió, se salta este paso.
3. **El recomendador busca recetas.** Con la lista de ingredientes y las preferencias del usuario, se consulta un grafo de conocimiento que puntúa qué platos encajan mejor.
4. **El *bot* devuelve la propuesta.** En segundos el usuario recibe en Telegram varias recetas que puede preparar con lo que tiene y que cumplen sus objetivos nutricionales.

Todo el sistema se despliega en una arquitectura *serverless* utilizando Google Cloud Functions, garantizando una alta disponibilidad, escalabilidad automática y una gestión optimizada de los costes operativos.

1.3. Objetivos

Objetivo general

Desarrollar un sistema integral, completamente desplegado en Google Cloud, que combine la detección automática de ingredientes, un motor de recomendación sobre grafos de conocimiento y una comunicación interactiva vía Telegram.

Sub-objetivos

Interfaz conversacional

- Implementar un *bot* de Telegram que gestione un breve cuestionario nutricional y permita al usuario elegir entre introducir los ingredientes manualmente o enviar una fotografía.

Visión por computador

- Realizar el *fine tuning* de un modelo YOLOv8 para la detección de ingredientes.
- Utilizar ese modelo para realizar inferencias sobre imágenes subidas por el usuario.

Recomendación basada en KGE

- Construir y enriquecer un grafo de conocimiento culinario.
- Desarrollar un motor de recomendación basado en grafos de conocimiento (Knowledge Graph Embeddings, en adelante KGE por sus siglas en inglés)

Seguridad, escalado y costes

- Aplicar el concepto de mínimo privilegio en *cloud* para el despliegue de la información mínimos privilegios en IAM, cifrado TLS/at-rest y logs centralizados.
- Aplicar el escalado automático y la facturación *pay-per-invoke* para la planificación de costos en soluciones autogestionada.

2. Gestión del proyecto

2.1. Modelo de ciclo de vida

En el curso del desarrollo del proyecto, se ha seguido una planificación estructurada en cascada e iterativa cuyas principales actividades, tareas y planificación fue definida mediante un diagrama de Gantt. Esta metodología, aunque más rígida que otras aproximaciones, proporciona una visión clara de las fases, hitos y dependencias críticas, facilita el cumplimiento y el seguimiento de los requisitos en el tiempo establecido. No obstante, lo anterior, esta aproximación metodológica permite el (i) desarrollo iterativo, necesario para soluciones experimentales basadas en IA como la que se requiere en este proyecto, así como (ii) la definición de tareas como Experimentación y desarrollo y su asignación de tiempo y recursos, permitiendo un trabajo iterativo en los modelos de detección de imágenes (YOLOv8) y del sistema de recomendación (basado en KGE).

Otro ejemplo de la capacidad de la flexibilidad fue el cambio en la arquitectura de despliegue. El proyecto arrancó con la hipótesis de desplegar los microservicios en Kubernetes (GKE), pero tras analizar la complejidad operativa, se migró a una arquitectura *serverless* basada en Cloud Functions Gen2 y Pub/Sub. La planificación fue lo suficientemente robusta para asimilar este pivote.

2.2. Planificación

La planificación se basa en una calendarización estructurada de las tareas. Se establecieron un plazo estimado para cada bloque de trabajo, así como la secuencia lógica que permite un desarrollo progresivo y coordinado de todas las actividades planteadas. Los bloques de trabajo y tareas que estructuran el proyecto son los que se indican en la tabla 1, siendo los principales:

- Fase inicial y definición del proyecto
- Diseño del sistema
- Desarrollo del *bot* y cuestionario
- Diseño del módulo de detección de ingredientes
- Diseño del módulo de recomendación
- Despliegue y configuración en Google Cloud
- Documentación

	Tarea	Inicio	Días	Final
Fase Inicial y Definición del Proyecto	Definición de objetivos y alcance del proyecto	10/10/2024	20	30/10/2024
	Investigación de fundamentos teóricos	30/10/2024	16	15/11/2024
	Revisión de soluciones posibles	15/11/2024	2	17/11/2024
	Selección preliminar de tecnologías y herramientas	17/11/2024	10	27/11/2024
Diseño del sistema	Especificación de requisitos funcionales y no funcionales	28/11/2024	13	11/12/2024
	Diseño de la arquitectura general del sistema	12/12/2024	13	25/12/2024
	Diseño de la interfaz del bot de Telegram	12/12/2024	13	25/12/2024
Desarrollo del Bot y Cuestionario	Implementación de la interfaz conversacional básica	02/01/2025	13	15/01/2025
	Desarrollo del cuestionario de preferencias	16/01/2025	8	24/01/2025
	Gestión de entrada de ingredientes (texto/imagen)	16/01/2025	8	24/01/2025
Diseño del Módulo de Detección de Ingredientes	Recopilación y preparación del dataset de ingredientes (Roboflow)	27/01/2025	18	14/02/2025
	Experimentación y desarrollo de pruebas de concepto en jupyter notebooks	17/02/2025	39	28/03/2025
	Entrenamiento inicial del modelo YOLOv8 (Transfer Learning)	31/03/2025	11	11/04/2025
	Evaluación y ajuste del modelo	14/04/2025	11	25/04/2025
Desarrollo del Módulo de Recomendación	Obtención del dataset de recetas	27/01/2025	18	14/02/2025
	Investigación librerías de KGE	17/02/2025	32	21/03/2025
	Preprocesamiento de recetas construcción del grafo(temas)	24/03/2025	11	04/04/2025
	Experimentación y desarrollo de pruebas de concepto en jupyter notebooks	07/04/2025	39	16/05/2025
	Entrenamiento del modelo KGE con PyKeen	19/05/2025	11	30/05/2025
	Evaluación y ajuste del modelo	02/06/2025	11	13/06/2025
Despliegue y Configuración en Google Cloud	Selección y aprovisionamiento de recursos en GCP	16/06/2025	2	18/06/2025
	Diseño e implementación de la arquitectura Pub/Sub	19/06/2025	6	25/06/2025
	Pruebas en la nube	26/06/2025	4	30/06/2025
Documentación	Preparación del repositorio GitHub	06/11/2024	3	09/11/2024
	Redacción de la memoria del proyecto	09/04/2024	60	30/06/2025

Tabla 1. Calendarización bloques de tareas

En la figura 1 se presenta el diagrama de Gantt correspondiente a la planificación inicial del proyecto. Este cronograma visual fue diseñado como una hoja de ruta para guiar el desarrollo, desglosando el trabajo en siete fases principales que se diferencian por colores, desde la definición de objetivos hasta la entrega final de la memoria. Una característica clave de este plan es que es optimista, con una fecha de finalización para finales de mayo de 2025. Se puede observar un alto grado de paralelismo en las tareas, especialmente en el desarrollo de los módulos de Detección (rosa) y Recomendación (azul), que se planificó empezar en paralelo para agilizar el proceso. Asimismo, el cronograma refleja la idea tecnológica original del proyecto, asignando un bloque de

tiempo específico en mayo para un "Posible despliegue en Kubernetes GKE". Para un análisis más detallado e interactivo de esta planificación, en el repositorio de GitHub se encuentra los archivos `diagrama_gantt_inicial.html` y `diagrama_gantt_real.html` dentro de la carpeta **planificación**. Este archivo puede abrirse directamente en cualquier navegador web para explorar las tareas y sus duraciones.

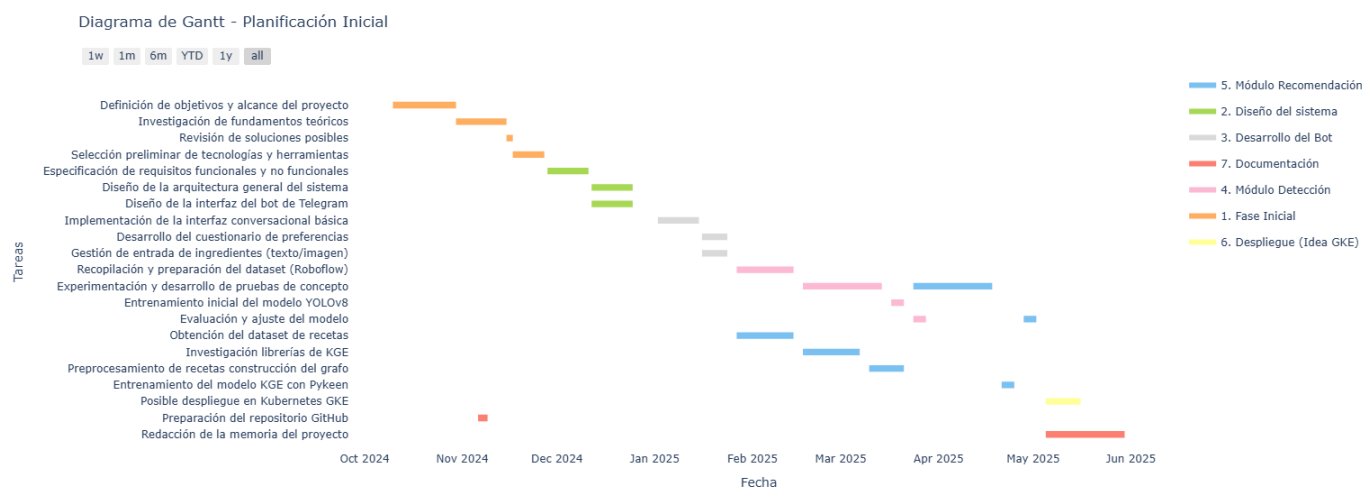


Figura 1. Diagrama de Gantt Inicial

La figura 2 muestra el Gantt de la planificación real, que extiende el cronograma hasta principios de Julio en contraste con la estimación inicial.

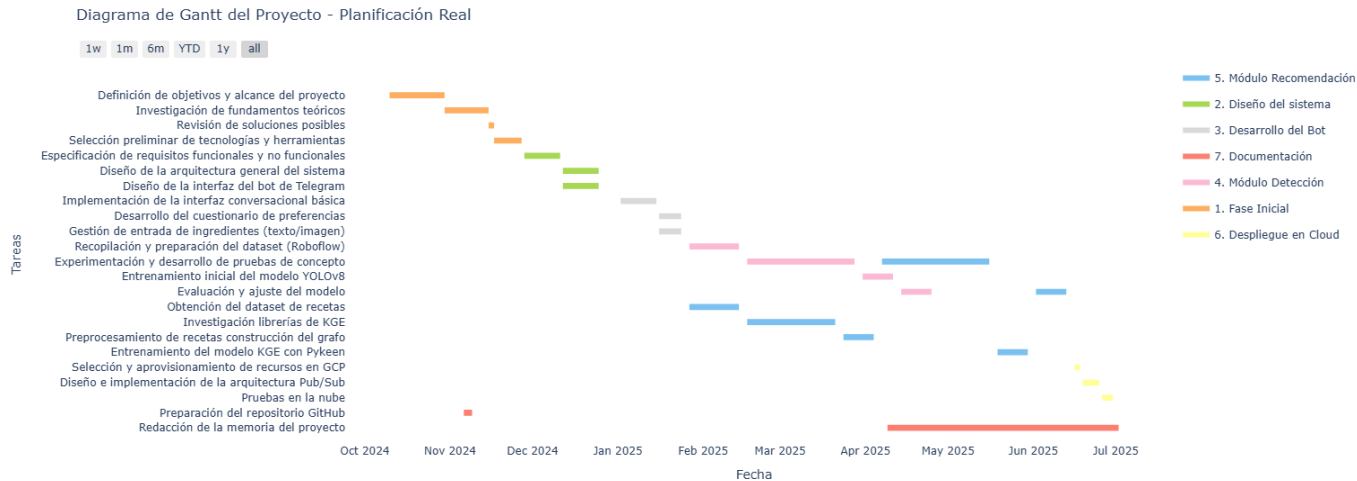


Figura 2. Diagrama de Gantt Real

Las desviaciones que se han dado como resultado el Gantt de la figura 2 han sido debidos a desajustes derivados de la investigación y experimentación. Así, inicialmente, se definieron las tareas y la duración estimada de las mismas (véase figura 1). Según esta estimación, el proyecto debía haber finalizado en mayo de 2025. Asimismo, se introdujeron aproximaciones y soluciones tecnológicas que, a la luz de la experiencia del proyecto, se mostraron menos óptimas para el desarrollo de éste que las que finalmente se adoptaron (como, por ejemplo, el despliegue usando Kubernetes en *cloud*). Sin embargo, tal y como muestra la figura 2, durante el desarrollo del proyecto, tareas de investigación o el ajuste los modelos llevaron más tiempo del inicialmente planificado, extendiendo la duración del proyecto hasta Julio. En este sentido, tras la experimentación e investigación llevada a cabo, se descartó la idea del despliegue basado en Kubernetes en *cloud* por su complejidad y coste, y se optó por una arquitectura *serverless* Google Cloud (Cloud Functions y Pub/Sub), que resultó ser una solución más eficiente y manejable, aunque este cambio también requirió tiempo de adaptación en el plan.

2.4. Recursos necesarios

Los recursos empleados en este proyecto se agrupan en cinco bloques: hardware local, software y herramientas de desarrollo, infraestructura *cloud*, validación y pruebas, y costes estimados.

- Hardware local
 - Procesador: 13th gen Intel Core i7-13700H x 20
 - Memoria RAM: 32 GB DDR5-4800
 - Unidad gráfica (GPU): GPU NVIDIA RTX 4070 Laptop (8 GB VRAM)
 - Disco duro: SSD NVMe 1 TB
- Software

Tabla 2. Recursos software

Propósito	Herramienta
Leguaje principal	Python 3.10
Detección de objetos	Ultralytics YOLOv8
Recomendación KGE	PyKeen 1.10.2
Análisis de datos	Pandas, NumPy
Bot de Telegram	Python-telegram-bot
CLI & CI	Gcloud SDK, Git + GitHub

- Infraestructura en Google Cloud Platform

Tabla 3. Recursos Google Cloud Platform

Recurso	Uso	Cuota Aproximada
X4 Cloud Functions Gen2	Lógica <i>serverless</i>	≈ 300 000 invocaciones/mes dentro del free-tier
x2 Pub/Sub topics	Orquestación asíncrona	Free-tier (<10 GB-e/s)
Firestore nativo (1 colección, <50 KB/mes)	Persistencia de preferencias del cuestionario	0,18 €/mes
Cloud Storage bucket (≈55 MB)	Modelos n	<0,01 €/mes

- Análisis de costes

Google Cloud Platform:

Actualmente el coste es de 0,22 € como se observa en la figura 3

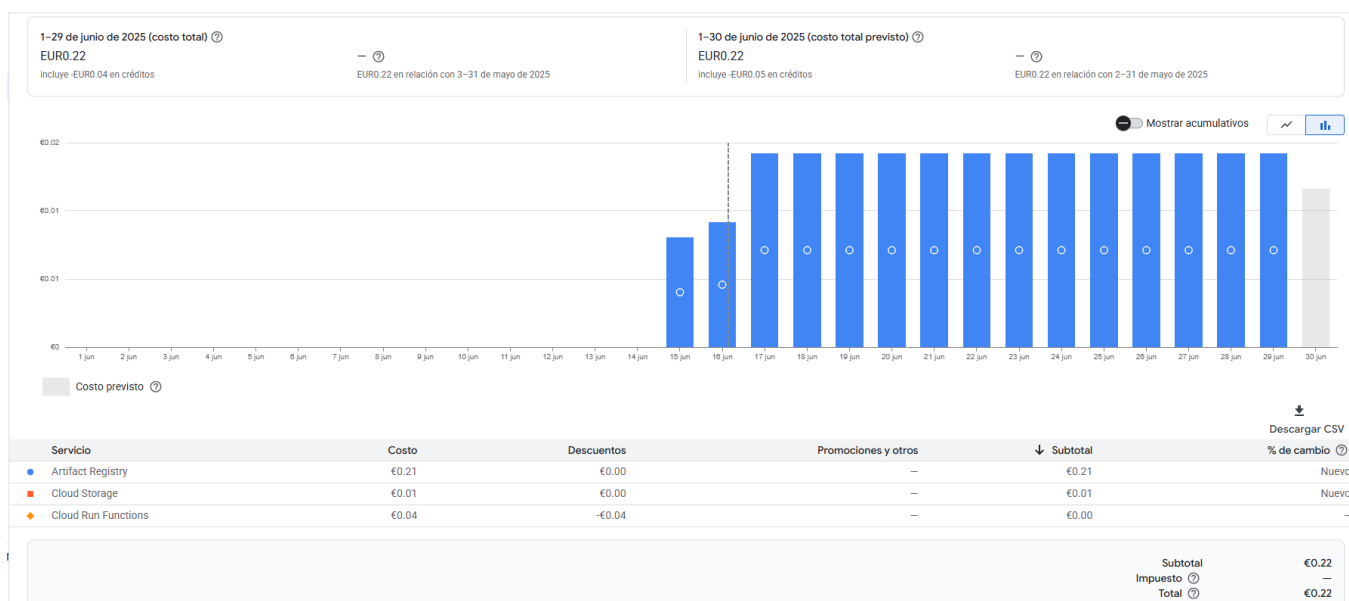


Figura 3. Google Cloud Platform costes actuales

Respecto al coste actual, que es de aproximadamente un céntimo diario desde su despliegue, es importante aclarar que este corresponde principalmente al almacenamiento de la imagen del contenedor en Artifact Registry, y no a la ejecución de las funciones en sí. Planteando un escenario hipotético en el que el servicio recibiera 1000 llamadas mensuales, el coste de ejecución de dichas llamadas seguiría siendo nulo. Esto se debe a que el nivel gratuito cubre mensualmente hasta, 2 millones de invocaciones, 180.000 segundos de tiempo de vCPU, 360.000 GiB-segundos de tiempo de memoria.

Coste Eléctrico:

Entre el 10 oct 2024 y el 30 jun 2025 se trabajó una media de 4 h GPU + 4 h tareas ligeras al día:

$$250 \text{ días} \times [(0,14 \text{ kW} \times 4 \text{ h}) + (0,05 \text{ kW} \times 4 \text{ h})] \approx 190 \text{ kWh}$$

$$190 \text{ kWh} \times 0,15 \text{ €/kWh} \approx 28,50 \text{ €}$$

2.5. Justificación tecnológica

Plataforma de despliegue Google Cloud

La elección de una arquitectura *cloud* y *serverless* facilita escalabilidad automática para gestionar cualquier pico de demanda, y abstracción respecto a despliegue y gestión de la infraestructura. El enfoque es más adecuado en una fase de experimentación y diseño de producto, al basarse en un modelo de pago por uso, lo que elimina los costes de infraestructura inactiva, y el centrarse en el diseño de la solución sin prestar atención a aspectos como gestión y diseño de la infraestructura subyacente. De este modo, no solo se reduce la carga operativa, sino que también otorga una gran flexibilidad para la evolución futura del sistema.

- Cloud Functions Gen2
 - *Serverless* puro: sin VMs ni clústeres que mantener escalado por invocación y facturación al milisegundo.
 - Despliegue inmediato con `gcloud functions deploy` integración nativa con Pub/Sub, Firestore y Cloud Storage.
 - Experiencia previa en mi empresa de prácticas, lo que acorta la curva de aprendizaje y reduce riesgos.
- Pub/Sub
 - Desacopla totalmente las etapas (Webhook, Detector, Recomendador Mensajero).
 - Entrega *at-least-once* y reintentos automáticos.
 - Sustituye a Kafka/RabbitMQ sin costes fijos ni complejidad operativa.

Detección de ingredientes – YOLOv8

- Arquitectura *state-of-the-art* con excelente relación precisión/velocidad y soporte oficial en ultralytics.
- Permite *transfer learning* rápido: 100 epochs en ≈ 15 h con mi GPU RTX 4070 (latencia ≈ 120 ms/CPU en Cloud).

- Peso del modelo (≈52 MB) encaja en el límite de 250 MB de Cloud Functions; no requiere GPU en producción.
- Alternativas (Detectron2, EfficientDet) ofrecen precisión similar, pero modelos más pesados o *pipeline* más complejo.

Motor de recomendación – PyKEEN

- PyKEEN tiene implementados los principales algoritmos para el entrenamiento de Knowledge Graph Embeddings (TransE, ComplEx, etc.) ya listos para usar.
- Aprende representaciones numéricas de ingrediente <-> receta <-> nutriente, captando relaciones latentes (p. ej. “pollo -> alto en proteína > receta”).
- El ranking se obtiene sumando los scores de cada relación nutricional (calorías, grasas, azúcar...), lo que hace muy fácil ajustar filtros o añadir nuevos nutrientes.
- Artefacto ligero: `modelo.pkl + triples.csv`, < 70 MB; se carga en caliente dentro del límite de disco de Cloud Functions Gen2.

Alternativas más simples

- Filtro colaborativo o aproximaciones estáticas, una tabla “ingrediente - recetas” con reglas fijas. Presenta el problema de *cold start* en el primer caso, o la imposibilidad de aprender nuevas combinaciones no prioriza por preferencia. K-Nearest Neighbours sobre vectores de nutrientes necesita escalar todas las recetas y no explica por qué sugiere cada plato.
- Estas opciones funcionan, pero PyKEEN aprende las relaciones del grafo y permite crecer (más entidades, más tipos de relación) sin reescribir la lógica de recomendación, ofreciendo sugerencias más precisas y justificables. Asimismo, sienta las bases de integraciones de datos externos para el incremento de relaciones semánticas o la integración de información.

Interfaz de usuario – Bot de Telegram

- Evita desarrollar y hospedar una web/app, Telegram aporta servidor, cuentas y notificaciones *push* gratis.

- BotFather + HTTP Webhook simplifican la autenticación y el enrutado de mensajes.
- Baja fricción para el usuario: no instala nada, sólo envía texto o fotos.
- Alternativas (web React, aplicación móvil) implican UI completa y mayor complejidad.

Persistencia – Firestore GCP

- Base de datos NoSQL gestionada con lecturas/escrituras de baja latencia y facturación por operación.
- Estructura documento - colección encaja con `users/{chat_id}` y evita esquemas rígidos o con demasiadas restricciones.
- Permisos controlados vía IAM y reglas de seguridad; sin necesidad de mantener instancias SQL.
- Alternativas gestionadas (Cloud SQL, BigQuery, Firebase RTDB): esquema rígido, mayor complejidad (sobre-engineered para <1kB por usuario).

Almacenamiento de artefactos – Cloud Storage

- Bucket smartfood-models guarda modelos y CSV
- Nivel gratuito cubre GB necesarios; versión única y ACL sencilla.
- Alternativas (Amazon S3, Git LFS, Artifact Registry): otro proveedor (rompiendo la política *single-vendor*) o límites de tamaño; orientadas a otros usos y añadirían credenciales innecesarias.

Herramientas de desarrollo

- Python 3.10: ecosistema maduro en IA, soporte oficial en Cloud Functions Gen2.
- Git + GitHub para control de versiones y CI/CD manual.
- Ultralytics, PyKEEN, pandas, google-cloud-sdk integran directamente con el stack GCP.
- Alternativas (Go, Node.js, Bitbucket/GitLab): lenguajes o sistemas de control de versiones no centrados en PyTorch; mi experiencia era en Python + GitHub.

Costes y sostenibilidad

- *Pay-per-use serverless* coste mensual cercano a 0 € en el nivel gratuito mientras no hay tráfico real. Sólo se paga por lecturas de Firestore y solicitudes HTTP del *bot*.
- Alternativas (GKE, VM permanentes): coste fijo alto y operación compleja frente al pago-por-uso *serverless*.

3. Análisis

3.1. Especificación de requisitos

En un escenario habitual, se deberán especificar, al menos, los siguientes tipos de requisitos:

Tabla 4. Requisitos

Ref.	Descripción
RF001	El sistema permitirá al usuario interactuar mediante el <i>bot</i> de Telegram sin necesidad de registro previo.
RF002	El sistema mostrará un cuestionario inicial para captar preferencias nutricionales (calorías, grasas, azúcar, sodio, proteína, carbohidratos).
RF003	El usuario podrá introducir ingredientes escribiendo texto; la lista se publicará en el tópico Pub/Sub ingredientes_detectados.
RF004	El usuario podrá enviar una fotografía; la función detectora (YOLOv8) extraerá los ingredientes y los publicará en el mismo tópico.
RF005	El sistema almacenará las preferencias del usuario en Firestore, asociadas a su <code>chat_id</code> .
RF006	La función recomendador consultará el grafo embebido (PyKEEN) y generará el top-k de recetas teniendo en cuenta los filtros multi-nutriente.
RF007	Si se produce un error interno, el sistema enviará un mensaje de fallo amigable y registrará el incidente en Cloud Logging.

3.2. Casos de Uso

Este artículo presenta el diseño del diagrama de caso de uso del sistema de recomendación de recetas, implementado a través de un *bot* de Telegram, que permite a los usuarios obtener recetas personalizadas según sus preferencias nutricionales e ingredientes disponibles. El modelo sigue los estándares de UML (*Unified Modeling Language* por sus siglas en inglés) y tiene como objetivo representar de manera clara y estructurada las funcionalidades del sistema desde el punto de vista del usuario.

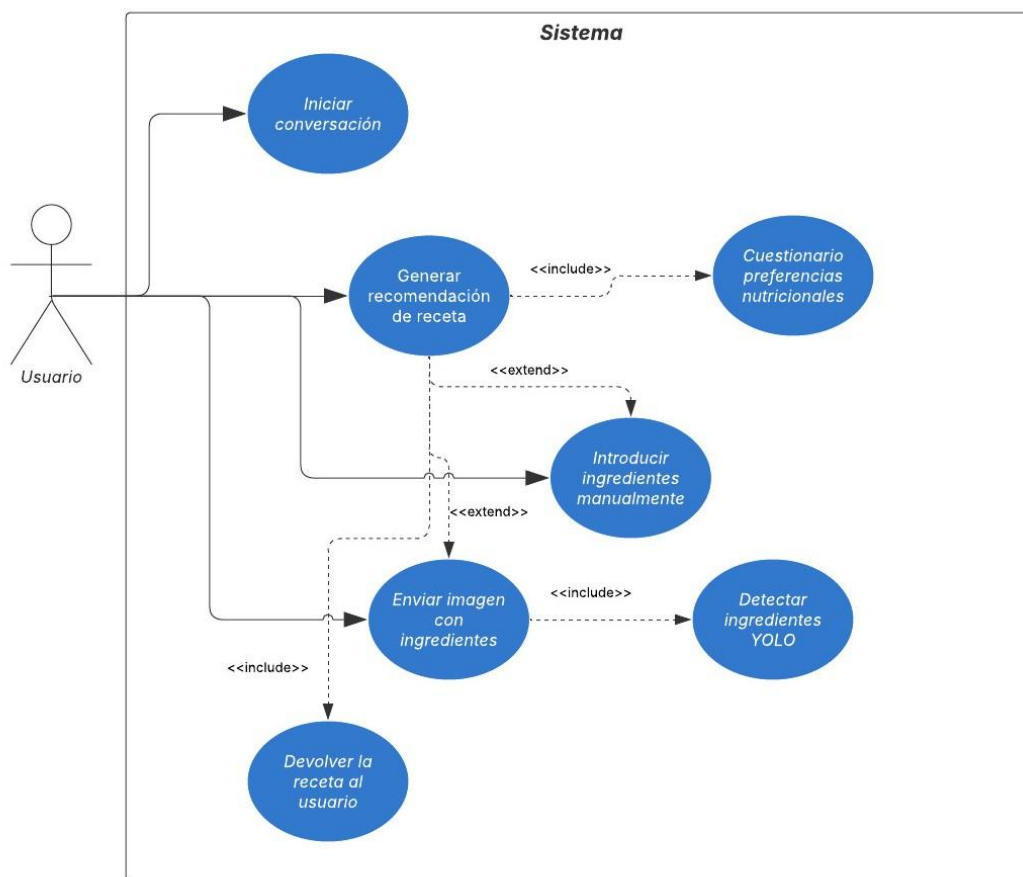


Figura 4. Casos de uso

1. Actor Principal

El “Usuario” de Telegram es el único actor del sistema, ya que es quien inicia y dirige toda la interacción. El sistema se implementa a través de un *bot* conversacional que automatiza todas las respuestas y flujos.

2. Caso de uso principal: Generar recomendación de receta personalizada

Este es el caso de uso central que agrupa todo el comportamiento funcional esperado del sistema. A partir de este, se desprenden diferentes subcasos que permiten modular el flujo y reflejar decisiones condicionales por parte del usuario.

3. Uso de relaciones <<include>> y <<extend>>

El diagrama emplea relaciones <<include>> y <<extend>> para expresar el comportamiento obligatorio y opcional del sistema:

- <<include>> se utiliza para representar comportamientos siempre necesarios para lograr el objetivo principal. Por ejemplo, el caso de uso Responder cuestionario de macronutrientes es un paso obligatorio para poder generar una recomendación, y por tanto se incluye siempre en el flujo principal.
- <<extend>> se emplea para modelar comportamientos opcionales o alternativos. En este contexto, el usuario puede elegir entre Enviar imagen con ingredientes o Introducir ingredientes manualmente. Ambos caminos extienden el caso de uso principal, pero solo uno será ejecutado por sesión, según la elección del usuario.

3.3. Definición de las interacciones y su secuencia en el sistema

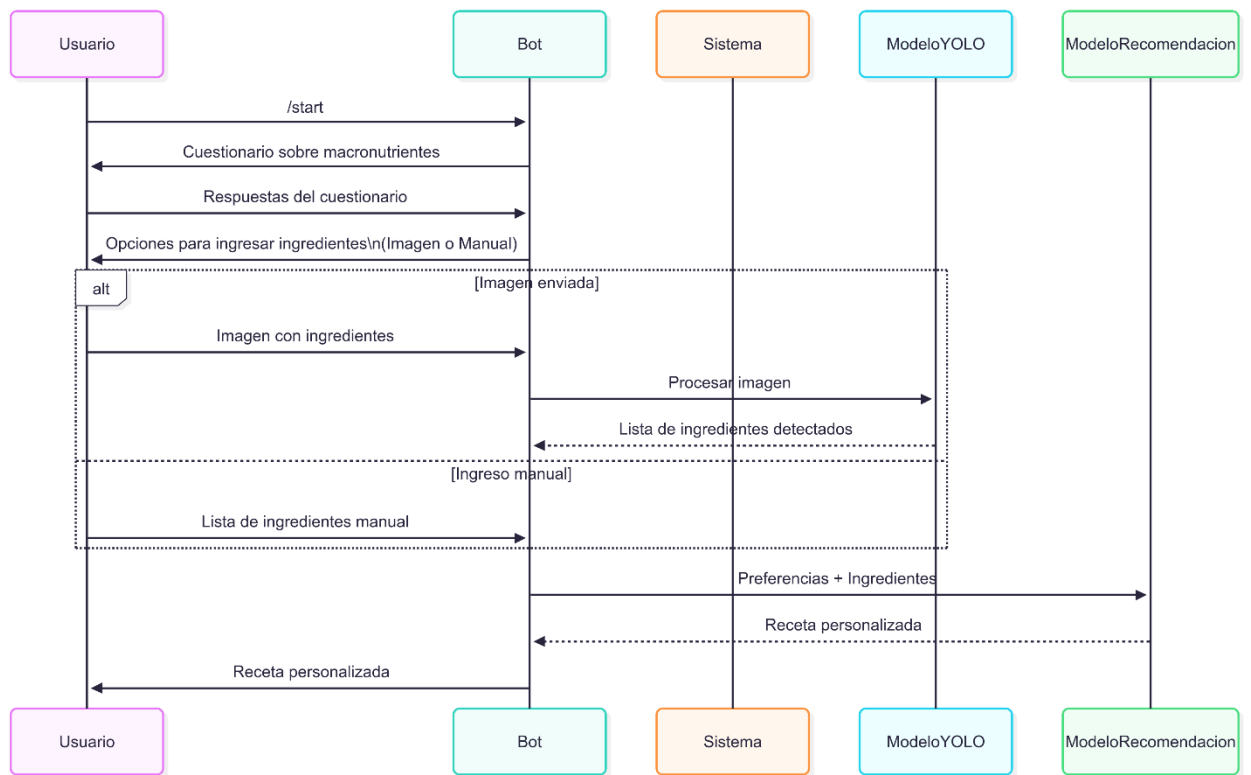


Figura 5. Diagrama de secuencia del sistema

El flujo de las interacciones en el sistema puede resumirse como sigue:

1. El usuario inicia una conversación con el *bot* mediante el comando `/start`.
2. El sistema presenta un cuestionario sobre preferencias de macronutrientes.
3. El usuario responde al cuestionario.
4. El sistema ofrece dos formas de introducir ingredientes:
 - a. Enviar una imagen, que será procesada mediante un modelo de visión por computadora (YOLOv8) para detectar los ingredientes.
 - b. Introducir los ingredientes manualmente.
5. Los ingredientes, junto con las preferencias del usuario, se envían a un modelo de recomendación.
6. El sistema genera y devuelve una receta personalizada al usuario.

4.Arquitectura del sistema

En esta sección se describe en detalle la solución completa desarrollada, un sistema capaz de generar recomendaciones nutricionales personalizadas a partir de imágenes o texto. Para comenzar, el apartado **4.1** establecerá el estado del arte y la justificación de las tecnologías seleccionadas, contextualizando las decisiones de diseño. A continuación, se expondrá el proceso de implementación de los dos modelos centrales de inteligencia artificial: por un lado, se detallará la preparación del modelo de recomendación basado en KGE en el apartado **4.2** y, por otro, el entrenamiento del modelo para el reconocimiento de ingredientes en imágenes en el apartado **4.3**. Finalmente, el apartado **4.4** integrará estos componentes al describir la arquitectura completa del sistema, tanto a nivel lógico, detallando el flujo de datos entre servicios, como a nivel físico, exponiendo la infraestructura de despliegue.

4.1 Contexto

Los sistemas de recomendación nutricional tienen como objetivo sugerir alimentos o recetas a los usuarios basándose en sus preferencias y restricciones. Los enfoques predominantes se fundamentan en el filtrado colaborativo, que modela las interacciones históricas entre usuarios e ítems. Sin embargo, estos métodos presentan limitaciones documentadas. El problema del arranque en frío (*cold-start problem*) impide generar recomendaciones para usuarios nuevos o sobre ítems con pocas interacciones. Adicionalmente, la falta de interpretabilidad de los modelos, a menudo basados en factorización de matrices, dificulta la explicación de por qué se sugiere un ítem concreto. Finalmente, su capacidad para integrar conocimiento de dominio externo, como la composición de recetas, datos nutricionales o relaciones taxonómicas entre ingredientes, es limitada.

Como alternativa, se han propuesto sistemas de recomendación basados en grafos de conocimiento (Knowledge Graphs, KG) para abordar estas limitaciones. Esta técnica

modela el dominio como un conjunto de entidades (usuarios, recetas, ingredientes) y las relaciones que las conectan. Los modelos de Knowledge Graph Embeddings (KGE) aprenden representaciones vectoriales de baja dimensión de estas entidades y relaciones, permitiendo inferir conexiones no explícitas y enriquecer las recomendaciones. El modelo RippleNet (Wang et al., 2018) fue uno de los primeros en proponer la propagación de las preferencias del usuario a través de las relaciones del grafo para explorar sus intereses. En el contexto alimentario, trabajos como FoodKG (FoodKG, s. f.) demuestran la construcción de un grafo de conocimiento a gran escala que integra información sobre alimentos, nutrientes, recetas y enfermedades para fundamentar las recomendaciones. De forma similar, Chen et al (Chen et al., 2023) en su trabajo Health-Aware Food Recommendation Based on Knowledge Graph and Multi-Task Learning, emplean un grafo de conocimiento y *embeddings* para ofrecer recomendaciones personalizadas. La implementación de estos modelos se realiza mediante librerías como PyKeen (Ali et al., s. f.), que proporciona un entorno estandarizado para su entrenamiento y evaluación.

El segundo componente de este sistema es un modelo de visión por computador para la identificación de ingredientes a partir de imágenes. Para esta tarea, se seleccionó la arquitectura YOLOv8. Los modelos de la familia YOLO (You Only Look Once) son detectores de objetos de una sola etapa (*single stage*) que destacan por su relación entre la precisión, medida en mAP (*mean Average Precision*), y la latencia de inferencia. La metodología de entrenamiento empleada es el aprendizaje por transferencia (*transfer learning*), que consiste en la especialización de un modelo pre-entrenado en un corpus de datos a gran escala (como COCO) para una tarea de detección específica, reduciendo así el coste computacional del entrenamiento desde cero. El uso de YOLOv8 para la identificación de alimentos está documentado en la literatura. Por ejemplo, Aseoche et al. (Aseoche et al., 2024) utilizan YOLOv8 en una aplicación para el reconocimiento de alimentos y la estimación de su valor nutricional. De manera análoga, Razali et al. (Razali et al., 2024) emplea el mismo modelo para la identificación de ingredientes como paso previo a un sistema de recomendación de recetas.

La arquitectura de este proyecto se justifica por la combinación de estas dos tecnologías para resolver problemas complementarios. El modelo de visión por computador YOLOv8 (en adelante, YOLO) se emplea para la extracción de entidades, los ingredientes, a partir de datos no estructurados (imágenes). Estas entidades son luego utilizadas como entrada para el sistema de recomendación basado en KGE, el cual modela las relaciones complejas entre dichos ingredientes, las preferencias del usuario y el corpus de recetas. Este enfoque multimodal permite que las recomendaciones se basen tanto en la disponibilidad de ingredientes del usuario como en el conocimiento semántico del dominio.

4.2 Preparación del recomendador basado en KGE

Inicialmente, en la fase experimental de este trabajo, se consideró usar la biblioteca AmpliGraph. Esta librería ofrecía un conjunto de herramientas robustas para la creación y evaluación de modelos de *embeddings* de grafos de conocimiento. Sin embargo, tras un análisis más detallado de las necesidades del proyecto y una comparativa, se determinó que PyKeen se ajustaba mejor, además contar el error.

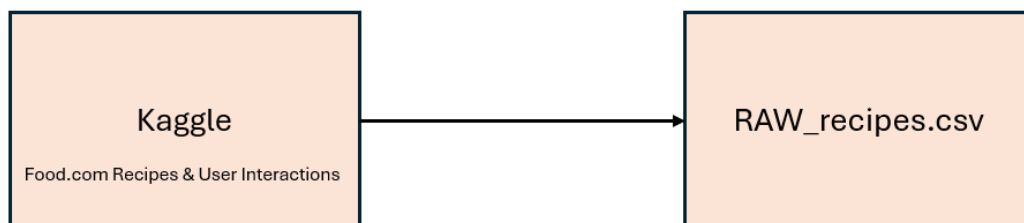


Figura 6. Preparación dataset KGE

El grafo de conocimiento se nutre del Food.com Recipes & User Interactions(*Food.Com Recipes and Interactions*, s. f.). El dataset reúne unas 230 000 recetas con buenos metadatos que encajan muy bien en este TFG.

- Existen dos tipos de relaciones principales:
 - Plato -> has_ingredient -> ingrediente
 - Plato -> has_nutrient -> low_/normal_/high_ nutriente
- La cadena `nutrition` agrupa siete valores de macronutrientes: calorías, grasas totales, azúcar, sodio, proteína, grasas saturadas y carbohidratos. Se transforman en entidades discretas (`low_`, `normal_`, `high_`) en función de su valor, estudiado previamente en el **Anexo I**. A partir de ese estudio se definen las relaciones `has_calories`, `has_total`, `has_sugar`.
- El campo `ingredients` define la lista textual de ingredientes. Tras normalizar (minúsculas, singularización, eliminación de cantidades) se cruza con el vocabulario de 80 ingredientes que el modelo YOLO es capaz de detectar: sólo los términos que aparecen en ambos mundos generan relaciones `has_ingredient`.
- El tiempo de preparación (`minutes`) todavía no se usa, pero quedaría disponible para filtrar platos rápidos en una versión futura de este sistema.
- Los campos `n_steps` y `steps` guardan el paso a paso de la receta, podría ser muy útil para que, en futuras iteraciones, el *bot* no sólo recomiende qué cocinar sino cómo hacerlo (e incluso limitar resultados a recetas con pocos pasos).

Tras la limpieza de ingredientes raros y la discretización nutricional, se obtienen más o menos 20 000 ternas. Este subconjunto balanceado es el que entra en PyKEEN para entrenar los *embeddings* descritos en los apartados siguientes.

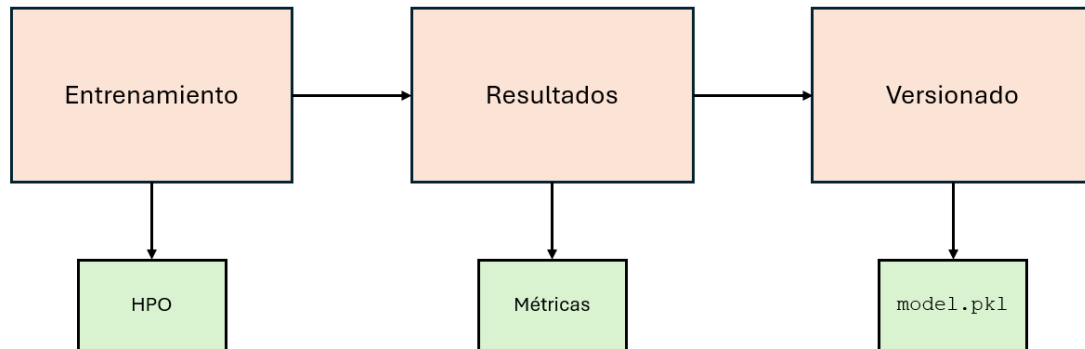


Figura 7. Pasos seguidos para la generación del recomendador basado en KGE

PyKEEN ofrece muchos algoritmos de *embedding*. Tras un Hyperparameter Optimization (HPO) preliminar con los modelos distMult, RotaE, TransE y ComplEx, detallado en el **Anexo II**, fue TransE el que logró los mejores resultados en el equilibrio entre las métricas de precisión y el tamaño del modelo. El modelo TransE (Translating Embeddings) definido en el trabajo de Bordes et al (*TransEInteraction — pykeen 1.11.2-dev documentation*, s. f.), se fundamenta en una intuición geométrica: para una terna válida (sujeto -h-, relación -r- y objeto -t-), el *embedding* de la cabeza h sumado al *embedding* de la relación r debería estar muy próximo al *embedding* de la cola t. La implementación de este modelo en PyKEEN se articula a través del módulo de interacción TransEInteraction, que es el responsable de calcular la puntuación de plausibilidad. Según la documentación oficial de este módulo, la función de puntuación mide la distancia entre $\mathbf{h} + \mathbf{r} \mathbf{y} \mathbf{t}$, para seguir la convención de que una puntuación más alta es mejor, se utiliza el negativo de la norma (distancia):

$$f(h, r, t) = -\|\mathbf{e}_h + \mathbf{e}_r - \mathbf{e}_t\|_p$$

En esta fórmula, los términos representan lo siguiente:

- $(\mathbf{h}, \mathbf{r}, \mathbf{y} \mathbf{t})$ son los vectores de *embedding* para la cabeza, la relación y la cola, respectivamente, en el espacio de los números reales (R).

- $\|\cdot\|_p$ denota la norma p , una función que mide la magnitud (o "distancia") del vector resultante.

Este enfoque de traslación es particularmente efectivo para modelar relaciones jerárquicas y composicionales, como las que se encuentran en nuestro grafo de conocimiento entre ingredientes y recetas.

El entrenamiento convergió en 100 épocas; los parámetros exactos quedan versionados en el archivo de configuración de PyKEEN. Los resultados obtenidos son los siguientes:

MRR	Hits@1	Hits@3	Hits@10	Mean Rank
0.5566	0.2808	0.9954	0.9968	6.798

Tabla 5. Resultados obtenidos del entrenamiento del modelo basado en KGE

Si bien la explicación de las métricas se detalla en el capítulo 5, la métrica HIT se refiere a la frecuencia (en porcentaje) con la que la entidad correcta es clasificada por el modelo dentro de un número k de las mejores respuestas posibles (ej. Top 10):

Cabe destacar que, aunque se ha alcanzado un valor de Hits@10 ≈ 0.99 , la precisión absoluta se resiente porque el grafo cubre muchas relaciones y escalas nutricionales. Incorporar más ternas y refinar las clases debería aumentar la fiabilidad de la sugerencia final

Finalmente, el artefacto final se empaqueta con PyKEEN:

- `torch.save(): model.pkl` (unos 30 MB)
- Ternas: `new_triples20.csv` (unos 6 MB)
- El fichero con las ternas y el `model.pkl` se suben al bucket `gs://smartfood-models`

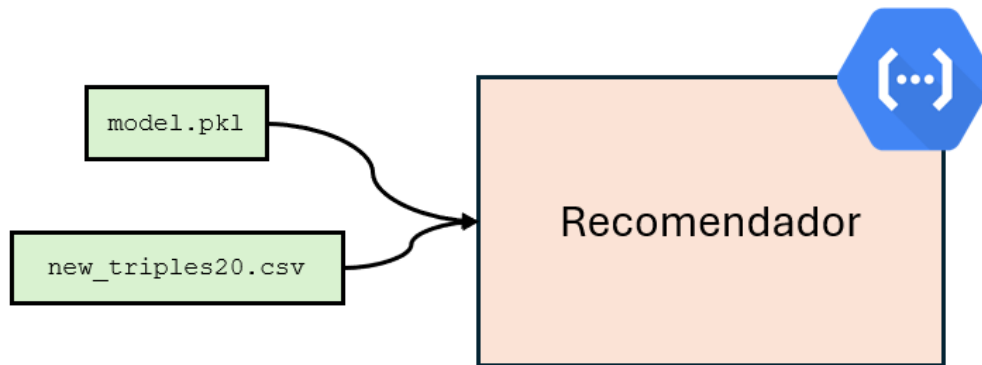


Figura 8. Despliegue de modelo KGE en Google Cloud

El microservicio Recomendador (Cloud Function Gen2) descarga `model.pkl` y el csv con las ternas al iniciarse y mantiene los objetos PyKeen en memoria. En tiempo de ejecución:

1. Suma los vectores-score de cada ingrediente recibido
2. Añade los vectores de las preferencias nutricionales del usuario
3. Ordena por puntuación y devuelve las k mejores recetas

Los detalles de cómo este microservicio se orquesta mediante Pub/Sub y se integra con el resto del sistema, se describen en la Arquitectura lógica y física (sección 4.4).

4.3 Preparación del modelo de reconocimiento de imágenes

La estrategia adoptada para el entrenamiento del modelo de reconocimiento de imágenes se basa en el aprendizaje por transferencia (*transfer learning*), una metodología estándar en visión por computador que reutiliza un modelo pre-entrenado como punto de partida para una tarea más específica. Como señalan Zhuang et al. (Zhuang et al., 2021), este enfoque reduce drásticamente la necesidad de un volumen masivo de datos y la alta capacidad computacional del entrenamiento desde cero. La técnica concreta empleada es el ajuste fino (*fine-tuning*), mediante la cual se conservan las capas iniciales de la red, que ya han aprendido a identificar características genéricas

como bordes o texturas y se reentrenan únicamente las capas superiores para que se especialicen en las nuevas clases de nuestro conjunto de datos de ingredientes.

Para este proyecto, y como ya se ha expuesto, se seleccionó el modelo YOLOv8. Al utilizar su versión pre-entrenada en el conjunto de datos COCO, se aprovecha una base robusta, lo que nos permite adaptar el modelo a nuestra tarea de forma mucho más eficiente (como ya se hace en trabajos similares como el Razali et al. (Razali et al., 2024)). Por tanto, en lugar de abordar el costoso proceso de desarrollar una red desde sus cimientos, se optó por especializar una herramienta ya optimizada. Las siguientes subsecciones detallarán el proceso específico de preparación de datos y ajuste fino del modelo.

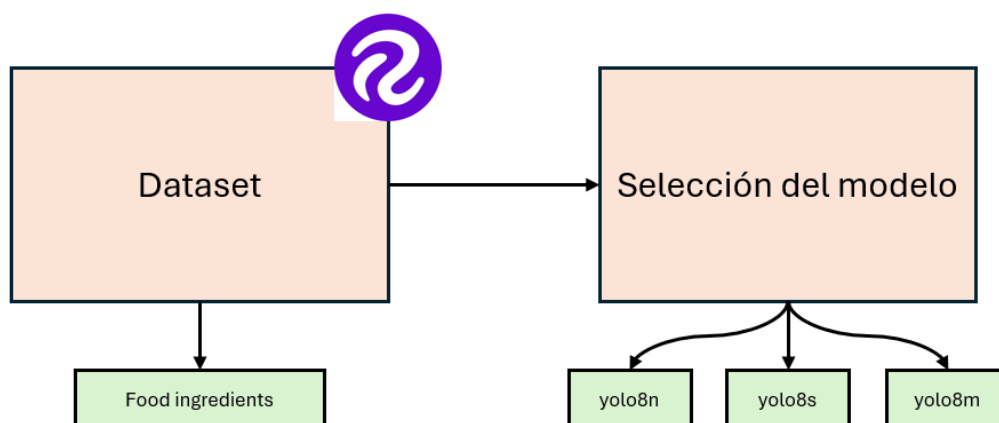


Figura 9. Preparación del dataset y selección de modelo YOLOv8

Para la formación del modelo de detección se utilizó el conjunto de datos público *FOOD-INGREDIENTS dataset Computer Vision Project*, disponible en la plataforma Roboflow(*FOOD-INGREDIENTS Dataset Dataset > Overview*, s.f.) Dicho dataset se compone de unas 4.000 imágenes que contienen ingredientes. El etiquetado ya venía proporcionado en el formato estándar de YOLO, el cual consiste en un fichero de texto (.txt) por cada imagen (.jpg) que contiene la clase y las coordenadas normalizadas de la caja delimitadora (*bounding box*).

Del total de clases disponibles en el dataset, se filtraron 80 que presentaban una correspondencia directa con las entidades definidas en el grafo de conocimiento de recetas (p. ej., carrot, bacon, potato). (hablar de si hice *data augmentation*) El conjunto de datos fue exportado con una división automática de 80% para entrenamiento y 20% para validación.

Para elegir el modelo final a ser utilizado en el sistema, se ha realizado un estudio comparativo de tres variantes de la arquitectura YOLO, que presentan unas diferencias entre precisión y eficiencia computacional: YOLOV8n (nano), YOLOV8s (small) y YOLOV8m (medium). Los tres modelos fueron analizados con un proceso de ajuste fino (*fine-tuning*) inicial de 50 épocas. El modelo yolov8n alcanzó un valor de mAP50 (mean Average Precision con un umbral de IoU de 0.5, según se explica en el Capítulo 5) de aproximadamente 0.82, con un tamaño en disco de solo 52 MB. Aunque los modelos de mayor tamaño (YOLOV8s y YOLOV8m) ofrecieron mejoras marginales (inferiores a 2 puntos porcentuales) en la precisión, su latencia de inferencia y consumo de memoria eran mayores.

Dado que el despliegue estaba previsto en un entorno de CPU de recursos limitados como Google Cloud Run, se concluyó que YOLOV8n ofrecía el balance óptimo entre rendimiento y coste computacional, por lo que fue seleccionado como el modelo final.

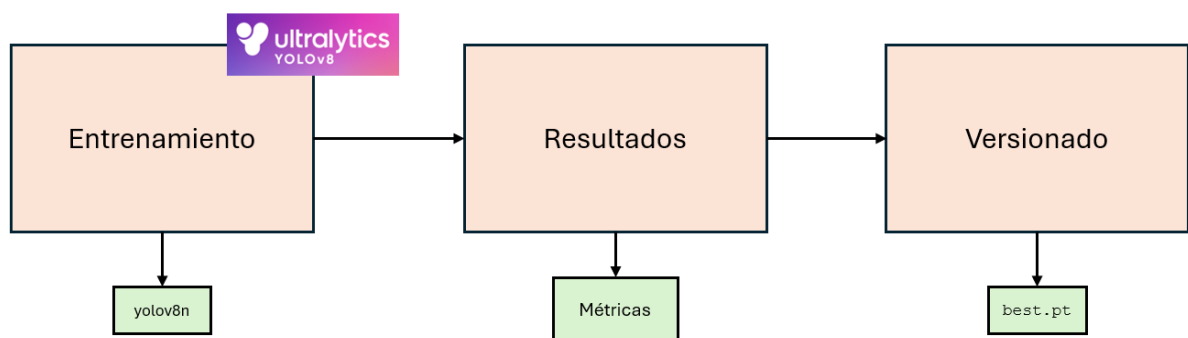


Figura 10. Pasos seguidos para el modelo de detección basado en YOLO

Una vez seleccionada la arquitectura, se procedió a un entrenamiento completo y más exhaustivo. La configuración final del proceso de entrenamiento utilizó los siguientes hiperparámetros:

- **Épocas:** 100
- **Tamaño del lote (Batch Size):** 16
- **Optimizador:** SGD (Descenso de Gradiente Estocástico) con una ratio de aprendizaje (lr) de 0.01 y un momento de 0.937.

Se incorporaron dos mecanismos clave para la regularización y eficiencia del entrenamiento: una estrategia de parada temprana (*early stopping*) con una paciencia de 20 épocas para evitar el sobreajuste, y un planificador de la ratio de aprendizaje de tipo coseno (`cosine LR scheduler`) para ajustar dinámicamente la tasa de aprendizaje.

El modelo final alcanzó una métrica mAP50 global de 0.924. Si bien la explicación de las métricas se detalla en el capítulo 5, la métrica mAP50 se refiere al rendimiento promedio de un modelo de detección de objetos, exigiendo un solapamiento mínimo del 50% entre la predicción y el objeto real para ser considerada un acierto. El análisis detallado por clase reveló un rendimiento robusto, con una media de precisión por clase que oscilaba entre 0.83 y 0.99. Tras la finalización del entrenamiento, se utilizó la función `export` de la librería Ultralytics para generar el artefacto de despliegue. Este proceso produce el fichero `best.pt`, que contiene los pesos optimizados del modelo y excluye los componentes del optimizador, reduciendo su tamaño a 52 MB y preparándolo para la inferencia.

Este artefacto fue subido a un bucket de Google Cloud Storage (`smartfood-models/yolo/best.pt`), siguiendo la práctica recomendada de desacoplar los modelos de la lógica de la aplicación.

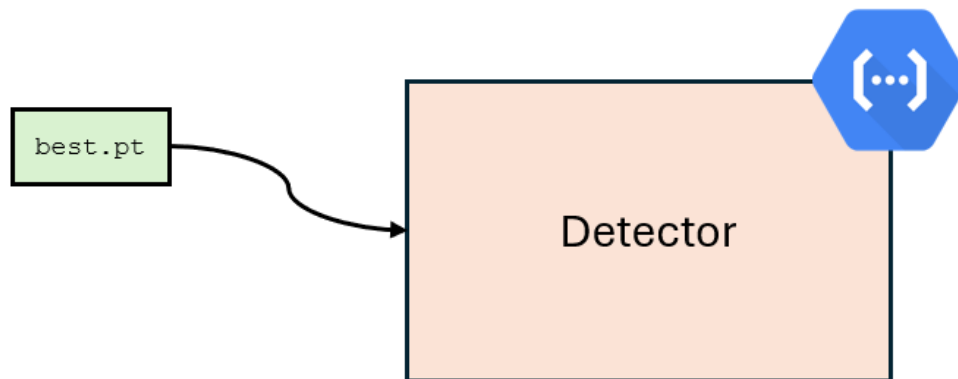


Figura 11. Despliegue del modelo de detección de imágenes en Google Cloud

El detector de ingredientes se desplegó como un microservicio encapsulado en una Cloud Function de segunda generación, utilizando el entorno de ejecución de Python 3.10. El flujo de operación es el siguiente:

1. El servicio recibe una petición HTTP que contiene la imagen del usuario enviada desde el Webhook de la aplicación principal.
2. El modelo `best.pt` se carga en memoria *lazy loading* la primera vez que se invoca la función, utilizando la clase Ultralytics. Para optimizar el rendimiento en ejecuciones posteriores, el modelo se almacena en caché en el directorio del entorno de la función, minimizando así la latencia asociada a los arranques en frío (*cold starts*).
3. Una vez procesada la imagen, la función publica una lista de los ingredientes detectados en formato JSON (p. ej., `{"ingredientes": ["potato", "bacon", ...]}`) en un *topic* de Google Cloud Pub/Sub denominado `ingredientes_detectados`, permitiendo una comunicación asíncrona y desacoplada con otros servicios del sistema.

Los detalles de cómo este microservicio se orquesta mediante Pub/Sub y se integra con el resto del sistema, se describen en la Arquitectura lógica y física (sección 4.4).

4.4 Arquitectura del sistema

Como se ha introducido en la sección 4.1, la arquitectura general del sistema sigue un enfoque de microservicios *serverless* donde cada componente se implementa como una Cloud Function independiente y los intercambios se coordinan mediante Pub/Sub. Esta combinación facilita el despliegue individual de cada servicio, el escalado automático y la comunicación asíncrona sin acoplamientos rígidos.

A muy alto nivel, el sistema se articula en cinco bloques:

- **Interfaz de usuario:** Un *bot* de Telegram que centraliza la interacción (texto o foto) y envía las peticiones hacia la nube.
- **Capa de orquestación:** Conjunto de Cloud Functions (Webhook, Detector, Recomendador, Mensajero) que procesan eventos de forma independiente y sin estado.
- **Bus de mensajería:** Dos *topics* Pub/Sub que intermedian los mensajes entre funciones y amortiguan ráfagas de tráfico.
- **Capa de datos:** Firestore almacena las preferencias de cada usuario; Cloud Storage aloja los artefactos de IA (modelos y triples).
- **Servicios externos:** Telegram Bot API y el SDK de Google Cloud que conectan la lógica interna con el exterior.

En los apartados 4.4.1 y 4.4.2 se describen, respectivamente, la arquitectura lógica (interacciones funcionales) y la arquitectura física (despliegue y recursos *cloud*) con mayor nivel de detalle.

4.4.1. Arquitectura lógica

El sistema adopta una arquitectura lógica por microservicios, en la que cada Cloud Function encapsula una única responsabilidad y se conecta de manera asíncrona mediante Pub/Sub, se muestran los componentes lógicos en la figura 12. Este diseño facilita el desarrollo y el despliegue independiente de cada módulo, reduce los acoplamientos y permite evolucionar o sustituir componentes sin afectar al resto.

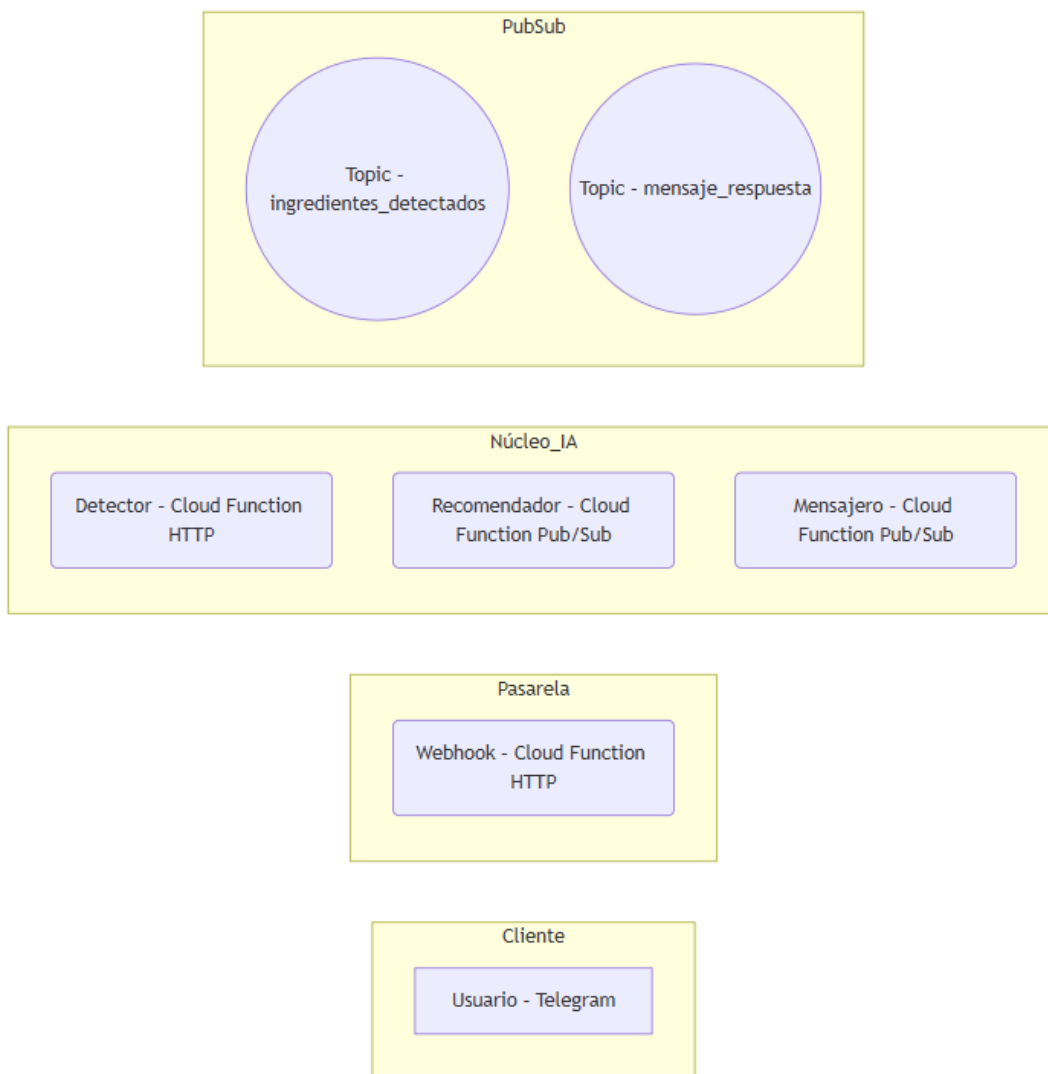


Figura 12. Componentes arquitectura lógica del sistema

El sistema adopta una arquitectura lógica basada en microservicios desacoplados donde cada módulo se especializa en una única responsabilidad funcional. Los elementos principales son:

Cliente – Telegram (TG)

- El punto de entrada y salida de la experiencia de usuario. Desde aquí se envían comandos (`/start`), texto con ingredientes o fotografías, y se reciben las respuestas finales.

Webhook (WEB)

Pasarela lógica que interpreta los mensajes entrantes.

- Gestiona el cuestionario de preferencias.
- Normaliza entradas (texto o imagen) y publica la lista de ingredientes en el bus de mensajería.

Bus de eventos (TOPIC1, TOPIC2)

- Dos *topics* lógicos separan el tráfico “peticiones” y “respuestas”. Actúan como capa de orquestación neutra: cualquier servicio puede publicar o suscribirse sin conocer la ubicación ni el lenguaje interno de los demás.

Núcleo IA

- **Detector (DET):** Consume imágenes y devuelve una lista de ingredientes detectados.
- **Recomendador (REC):** Combina ingredientes + preferencias con el grafo de conocimiento y calcula el ranking de recetas.
- **Mensajero (MSG):** Formatea la respuesta humana y la publica de vuelta al canal del usuario.

Esta **segmentación funcional** facilita la evolución independiente de cada componente: por ejemplo, se podría sustituir el modelo de detección de imágenes por otro sin

modificar el recomendador, o ampliar el grafo de conocimiento sin modificar el Webhook. La capa de eventos garantiza que los mensajes circulen en orden lógico, mientras que la responsabilidad de cada servicio se mantiene clara y aislada. En la figura 13 se muestra cómo estos componentes colaboran dentro del flujo lógico completo.

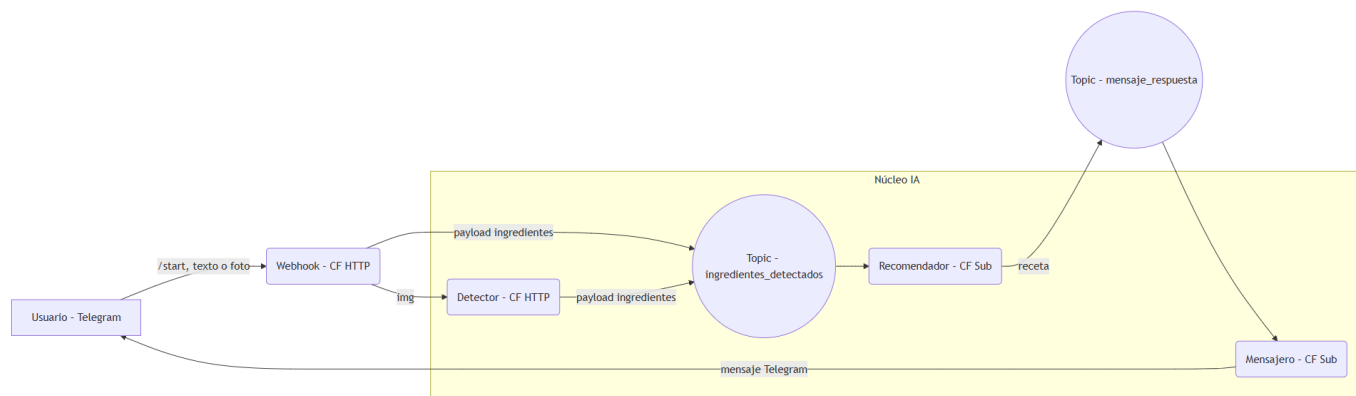


Figura 13. Flujo arquitectura lógica

4.4.2. Arquitectura física

La arquitectura física del sistema resumida en la figura 14. Arquitectura desplegada en Google Cloud se apoya en servicios *serverless* de Google Cloud Platform.

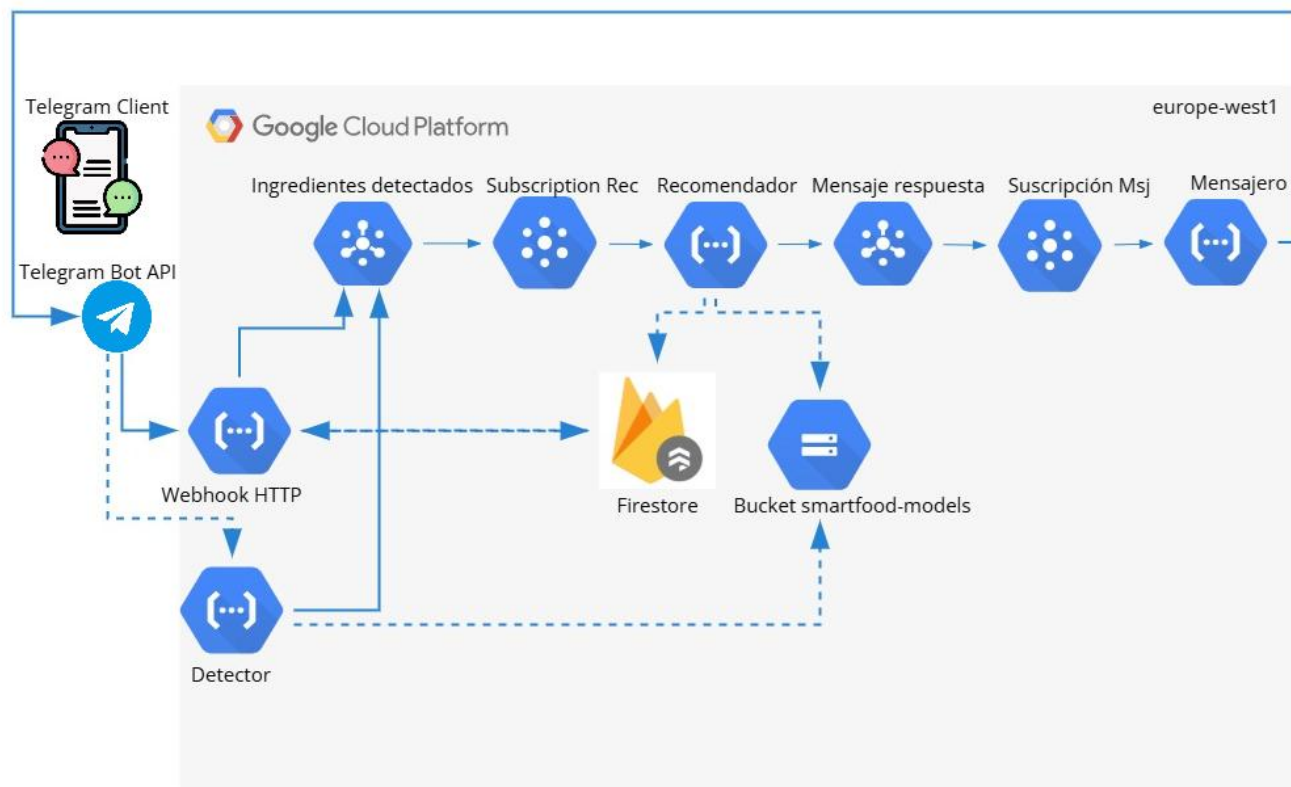


Figura 14. Arquitectura física del sistema

- **Capa de acceso**

El **cliente de Telegram** (móvil o PC) interactúa con el Telegram Bot API, que actúa como puerta de entrada a la nube. Todos los mensajes, ya sean texto o imágenes, llegan a la **Cloud Function “Webhook HTTP”** a través de una llamada HTTPS autenticada por el propio API de Telegram.

- **Plano de mensajería**

La función Webhook publica los ingredientes / preferencias en el **topic Pub/Sub** `ingredientes_detectados`. Este patrón *publish-subscribe* desacopla por completo la entrada de datos del procesamiento posterior, permitiendo picos de tráfico sin perder mensajes (garantía *at-least-once* de Pub/Sub).

- **Procesamiento**

- **Detector** (opcional) descarga la imagen, lee el modelo YOLOv8 desde Cloud Storage y publica los ingredientes detectados en el mismo *topic*.
- **Suscripción** `Rec` extrae cada mensaje y activa la función Recomendador, que carga los *embeddings* y el CSV de triples desde el *bucket* `smartfood-models` y consulta Firestore para recuperar las preferencias del usuario.
- Las recetas recomendadas se publican en el *topic* `mensaje_respuesta`.

- **Entrega**

- **Suscripción** `Msj` desencadena la función **Mensajero**, que formatea la lista de recetas y las envía al usuario vía Telegram Bot API, cerrando el ciclo.

- **Almacenamiento**

- **Firestore** (modo nativo) guarda un documento por chat (`users/{chat_id}`) con preferencias y estado.
- **Cloud Storage** almacena los archivos pesados: `model.pkl` (KGE), `triples.csv` y `best.pt` (YOLO). El nivel gratuito cubre el tamaño ($\approx$ 100 MB en total).

4.5 Bot

El punto de entrada al sistema es un *bot* de Telegram registrado con *BotFather* y expuesto mediante HTTP Webhook. Tal como se razonó en la sección 2.4 (Justificaciones tecnológicas), esta elección evita desplegar y mantener un *front-end* propio: Telegram ya ofrece la infraestructura de mensajería, entrega *push* y gestión de usuarios sin coste añadido. Al recibir el comando `/start`, el *bot* inicia un asistente de siete preguntas (calorías, grasas totales, azúcar, sodio, proteína, grasas saturadas y carbohidratos) y guarda las respuestas en Firestore (`users/{chat_id}`), de modo que las preferencias nutricionales se consultan y actualizan en $O(1)$. Cuando el usuario queda en estado *ready*, el *bot* acepta texto con ingredientes o fotos:

- Si envía texto, el webhook tokeniza la cadena, normaliza cada ingrediente y publica un *payload* JSON en `ingredientes_detectados`.
- Si envía una foto, reenvía la imagen al microservicio Detector; una vez YOLOv8 devuelve la lista de ingredientes, publica el mismo *payload*.

El *bot* vive en una Cloud Function Gen2 (`webhook/`) implementada con *functions-framework*, de arranque < 150 ms y despliegue continuo vía gcloud. El *endpoint* HTTP se verifica automáticamente con Telegram mediante un simple `POST`. Cuando el Mensajero publica las recetas en `mensaje_respuesta`, el *bot* las formatea en Markdown y las entrega con `sendMessage`. Todo el ciclo se mantiene *stateless*, lo que permite que las funciones escalen a cero instancias sin tráfico. Esta integración reduce al mínimo la fricción para el usuario, basta con abrir Telegram, mientras que alternativas como una web React o una aplicación móvil implicarían *hosting*, certificados TLS y ciclos de publicación en tiendas, incrementando complejidad y coste sin aportar valor adicional.

4.6 Seguridad

La estrategia de seguridad de este proyecto no se ha centrado en el desarrollo de componentes a medida, ya que no era el objetivo del trabajo, sino en el uso de las características de seguridad que ofrecen por defecto las plataformas seleccionadas: Google Cloud Platform y Telegram.

La mayor parte de la protección del sistema se hereda directamente de la infraestructura de Google Cloud. Por defecto, la plataforma cifra automáticamente todos los datos, tanto en tránsito (con TLS) como en reposo, incluyendo los artefactos del modelo almacenados en Cloud Storage. Asimismo, cada Cloud Function se ejecuta en un entorno aislado (*sandbox*), lo que limita su impacto potencial. Para el control de accesos, se ha seguido la práctica recomendada de asignar a cada servicio roles de IAM predefinidos con los permisos mínimos necesarios para operar, sin conceder privilegios excesivos (*least privilege*).

A nivel de aplicación, la principal medida de seguridad implementada activamente fue la gestión de secretos. Para proteger credenciales sensibles como el token del *bot*, se configuraron como variables de entorno en Cloud Functions. Esta práctica estándar asegura que ninguna clave o secreto quede expuesto en el código fuente ni en el repositorio de control de versiones. La comunicación con el usuario final se apoya a su vez en el cifrado que proporciona la propia API de Telegram.

4.7 Referencia al repositorio de software

El código fuente del sistema se encuentra en el siguiente repositorio de GitHub:

Enlace: <https://github.com/Jmartinezc-rgb/SmartFood>

En el caso de que el tribunal quiera probar el *bot* puede acceder al siguiente enlace, el único requisito es tener una cuenta de Telegram:

Enlace: https://t.me/javi_mc_bot

5. Validación del sistema

5.1. Plan de pruebas

Para validar la solución se definieron cuatro pruebas de integración ejecutadas íntegramente desde la CLI de Google Cloud y llamadas `curl`. Con ello se evita depender de plataformas externas de pago y se reproducen las peticiones reales que llegarán al sistema en producción. A continuación, se expone el alcance y el resultado esperado de cada prueba:

Prueba 1: Camino completo (entrada en texto)

Se envían ingredientes como texto al Webhook.

Recorrido: Webhook -> Recomendador -> Mensajero -> Telegram.

Resultado: el usuario recibe en su chat las cinco recetas con mayor puntuación, confirmando que la ruta principal funciona de extremo a extremo.

Prueba 2: Camino completo (entrada en imagen)

Se publica la URL de una fotografía con alimentos, el Webhook reenvía la imagen al Detector YOLO.

Recorrido: Webhook -> Detector -> Pub/Sub -> Recomendador -> Mensajero -> Telegram.

Resultado: el Detector registra en Cloud Logging los ingredientes detectados y el usuario recibe las recetas pertinentes, verificándose la rama de procesamiento de imágenes.

Prueba 3: Resiliencia del servicio Mensajero

Se detiene temporalmente el servicio Cloud Run que implementa Mensajero (limitándolo a 0 instancias), se envía una solicitud y, pasados unos segundos, se reactiva el servicio.

Objetivo: demostrar que Pub/Sub conserva el mensaje mientras el consumidor está inactivo y lo entrega en cuanto vuelve a estar disponible.

Resultado: tras la reactivación el usuario recibe la respuesta sin pérdida de datos, evidenciando durabilidad de cola y reintento automático.

Prueba 4: Manejo de entrada vacía

Se envía una cadena en blanco.

Objetivo: comprobar la validación de la función Webhook y evitar publicaciones inútiles.

Resultado: el *bot* responde “No he reconocido ingredientes” y no se genera ningún mensaje en el *topic* ingredientes_detectados, confirmando el control de errores del lado del servidor.

5.2. Evaluación del sistema

La evaluación del sistema se descompone en dos fases, correspondiendo a los dos componentes principales desarrollados: el modelo de *embeddings* de grafo de conocimiento (KGE) para la recomendación y el modelo de detección de objetos para la identificación de ingredientes. En esta sección se definen y justifican las métricas seleccionadas para cuantificar el rendimiento de cada subsistema de manera objetiva.

Métricas de evaluación para el modelo KGE

La tarea principal del modelo KGE en este proyecto es la **predicción de enlaces** (*link prediction*). Específicamente, dado un par de entidades y una relación (p. ej., una tupla incompleta como (receta_A, tiene_ingrediente, ?)), el modelo debe ser capaz

de puntuar y clasificar todas las posibles entidades candidatas para completar la tupla, posicionando la entidad correcta (*ground truth*) en el rango más alto posible. Para evaluar esta capacidad de clasificación, se ha seleccionado un conjunto de métricas estándar en grafos de conocimiento.

- **Mean Rank (MR):** Es la media aritmética de la posición (rango) en la que se encuentra la entidad correcta en la lista de candidatos para todas las tuplas de evaluación. Un valor más bajo indica un mejor rendimiento, ya que significa que, en promedio, el modelo sitúa las respuestas correctas más cerca de las primeras posiciones. Es muy sensible a valores atípicos (una sola predicción con un rango muy alto puede afectar negativamente la media), lo que justifica la necesidad de otras métricas complementarias.
- **Hits@k:** Esta métrica mide el porcentaje de veces que la entidad correcta se encuentra dentro de las k primeras posiciones de la lista de resultados. En este trabajo se utilizan Hits@1, Hits@3 y Hits@10.
 - **Hits@1** representa la precisión del modelo en su predicción más probable; es la fracción de veces que la respuesta correcta es exactamente la primera de la lista.
 - **Hits@3 y Hits@10** son indicadores de la utilidad práctica del modelo en un escenario de recomendación real, donde un usuario estaría dispuesto a revisar una lista corta de sugerencias. Miden la capacidad del modelo para incluir la solución correcta dentro de un conjunto reducido de opciones.
- **Mean Reciprocal Rank (MRR):** Es la media de los rangos recíprocos de las predicciones correctas. Para cada tupla de test, si la entidad correcta es clasificada en la posición r , su contribución al MRR es de $1/r$. A diferencia del Mean Rank, el MRR es menos sensible a los valores atípicos y pondera de forma más significativa las predicciones con un rango alto. Un modelo que clasifica consistentemente las respuestas correctas en las primeras posiciones obtendrá una puntuación cercana a 1.

Métricas de evaluación para el modelo de detección

La evaluación de un modelo de detección de objetos como YOLOv8 es multifacética, ya que debe medir tanto la capacidad del modelo para clasificar correctamente un objeto como su precisión al localizarlo espacialmente mediante una caja delimitadora (*bounding box*). Las métricas fundamentales se basan en el concepto de Intersección sobre Unión (IoU), que mide el grado de solapamiento entre la caja predicha por el modelo y la caja real (anotada manualmente), y se calcula como el área de la intersección dividida por el área de la unión. Un umbral de IoU (p. ej., 0.5) se utiliza para determinar si una detección es un Verdadero Positivo (TP) o un Falso Positivo (FP). A partir de aquí, existen métricas más complejas:

- **Precisión (Precision):** Mide la proporción de detecciones correctas sobre el total de detecciones realizadas por el modelo ($TP / (TP + FP)$). Una alta precisión indica que el modelo no genera muchos falsos positivos.
- **Recall (Exhaustividad):** Mide la proporción de objetos reales que el modelo fue capaz de detectar ($TP / (TP + FN)$), donde FN son los Falsos Negativos. Un alto recall indica que el modelo no omite la detección de muchos objetos presentes en la imagen.
- **Average Precision (AP):** La precisión y el *recall* están inversamente relacionados; un modelo puede mejorar una a costa de la otra. La curva de Precisión-Recall visualiza este compromiso. El AP se define como el área bajo esta curva y proporciona un valor único que resume el rendimiento de un modelo para una única clase de objeto.
- **mean Average Precision (mAP):** Es la métrica principal para la evaluación global de modelos de detección de objetos. Se calcula como la media del AP a través de todas las clases de objetos del dataset. El mAP proporciona una única cifra que resume de forma equilibrada el rendimiento general del modelo.

La elección de mAP50 como métrica principal se justifica porque para la aplicación práctica de identificar ingredientes, no se requiere una localización extremadamente

precisa de la caja delimitadora, un solapamiento del 50% se considera suficiente para una detección correcta.

6. Conclusiones y líneas futuras

6.1. Conclusiones

El objetivo del proyecto era proporcionar una respuesta rápida y personalizada a la pregunta “¿Qué puedo cocinar con lo que tengo?”, integrando en un único sistema la detección automática de ingredientes, un motor de recomendación nutricional y una interfaz de mensajería accesible. Tras la evaluación inicial de distintos modelos de visión por computador (YOLOV8m, YOLOV8s, YOLOV8n), de varios algoritmos de KGE (TransE, DistMult, RotatE), así como del análisis de las alternativas de despliegue (Kubernetes, App Engine, Cloud Run), se optó por montar una solución *serverless* basada en Cloud Functions Gen2 y Pub/Sub principalmente porque minimiza costes operativos y elimina la gestión de infraestructura.

Los resultados de ambos modelos han demostrado ser apropiados para un prototipo académico. Desde el punto de vista de la integración de extremo a extremo el *pipeline* funciona de forma asíncrona, tolera fallos transitorios y escala bajo demanda, confirmando la viabilidad de la propuesta. El cuestionario inicial implementado en PyKEEN se combina con el filtrado de ingredientes y elementos nutricionales, lo que permite entregar recomendaciones accesibles y alineadas con las preferencias del usuario. En conjunto, el proyecto prueba que es posible ofrecer servicios de recomendación nutricional de nueva generación combinando visión por computador, KGE y mensajería instantánea sobre una plataforma *cloud* ligera y económica.

6.2. Líneas futuras

Se identifican varias líneas de trabajo futuro para la mejora y evolución del sistema que pueden agruparse en tres áreas principales:

El modelo de visión por computador: el rendimiento del modelo de detección YOLOv8 se puede optimizar. Durante la evaluación se observó que la precisión disminuía en clases con una representación minoritaria en el *dataset* (aquellas con ≤ 10 instancias). La línea de trabajo sería la mejora del conjunto de datos, aumentando la cobertura de estas clases minoritarias y aplicando *data augmentation*. Adicionalmente, se podría experimentar con variantes de mayor tamaño de la arquitectura YOLOv8 o explorar modelos de segmentación de instancias para obtener una delimitación más precisa de los ingredientes.

El grafo de conocimiento: constituye la base del recomendador y su ampliación con fuentes externas y nuevas relaciones enriquecería la calidad de las sugerencias. Se propone expandirlo para incluir recetas de múltiples cocinas internacionales, incorporar valores nutricionales actualizados desde fuentes de datos verificadas y añadir nuevas relaciones que modelen los procesos de preparación culinaria. Estas medidas mejorarían la precisión de las recomendaciones y habilitarían el desarrollo de estrategias avanzadas de trazabilidad, permitiendo justificar por qué se sugiere una receta concreta.

La infraestructura de producción: en el ámbito de la puesta en producción y MLOps, se intentaría reducir la latencia y mejorar la mantenibilidad. Se considera la migración de los servicios de inferencia a instancias de Cloud Run con soporte para GPU, lo que aceleraría drásticamente el procesamiento de imágenes y la consulta de *embeddings*. A largo plazo, se propone cerrar el ciclo de MLOps mediante la integración de un sistema de métricas continuas y la creación de un panel de observabilidad que monitorice el rendimiento de los modelos en tiempo real y detecte posibles derivas (*model drift*).

Bibliografía

Ali, M., Hoyt, C. T., & Domingo-Fernandez, D. (s. f.). *Predicting Missing Links Using PyKEEN*.

Aseoche, N. G. B., Al-Ismail, S. I. B., & Dannug, E. J. M. (2024). Nutrieye: An Android-Based Food Nutritional Value Recognition App Using Yolov8 Algorithm. *IC-ITECHS*, 5(1), Article 1. <https://doi.org/10.32664/ic-itechs.v5i1.1706>

Chen, Y., Guo, Y., Fan, Q., Zhang, Q., & Dong, Y. (2023). Health-Aware Food Recommendation Based on Knowledge Graph and Multi-Task Learning. *Foods*, 12(10), Article 10. <https://doi.org/10.3390/foods12102079>

Food.com Recipes and Interactions. (s. f.). Recuperado 2 de julio de 2025, de <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>

FOOD-INGREDIENTS dataset Dataset > Overview. (s. f.). Roboflow. Recuperado 1 de julio de 2025, de <https://universe.roboflow.com/food-recipe-ingredient-images-Ognku/food-ingredients-dataset>

FoodKG. (s. f.). Recuperado 1 de julio de 2025, de <https://foodkg.github.io/>

Melore, C. (2021, octubre 20). Kitchen nightmare: 1 in 3 agree pandemic has dulled their love for cooking. *Study Finds*. <https://studyfinds.org/pandemic-cooking-fatigue/>

- Razali, M. R. M., Almarzuki, H. F., Bahar, R. A., & Abdullah, N. A. S. (2024). *Recipe Recommendation based on Food Ingredients Recognition using Deep Learning*. 555-564. https://doi.org/10.2991/978-94-6463-589-8_52
- Team, S. (2019, junio 18). Spoon Guru Study: Consumers Need Help Identifying Healthy Choices. *The Shelby Report*. <https://theshelbyreport.com/2019/06/18/spoon-guru-study-consumers-need-help/>
- TransEInteraction—Pykeen 1.11.2-dev documentation*. (s. f.). Recuperado 1 de julio de 2025, <https://pykeen.readthedocs.io/en/latest/api/pykeen.nn.modules.TransEInteraction.html>
- Wang, H., Zhang, F., Wang, J., Zhao, M., Li, W., Xie, X., & Guo, M. (2018). RippleNet: Propagating User Preferences on the Knowledge Graph for Recommender Systems. *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, 417-426. <https://doi.org/10.1145/3269206.3271739>
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2021). A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE*, 109(1), 43-76. <https://doi.org/10.1109/JPROC.2020.3004555>

Anexo I – Análisis de la distribución de nutrientes

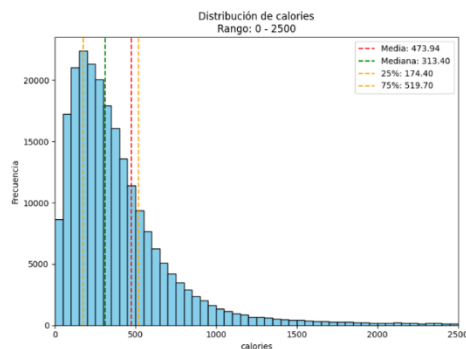
I.1 Introducción

Se realiza un estudio estadístico sobre la distribución de los valores nutricionales de las recetas, con el fin de definir umbrales que permitan clasificar cada receta como baja, normal o alta en un determinado nutriente. Este análisis respalda el sistema de recomendaciones personalizado.

I.2. Visualización y análisis descriptivo

A continuación, se presentan las gráficas de distribución para cada variable, acompañadas de los valores estadísticos más relevantes: media, mediana y percentiles 25 y 75. Se observan en todos los casos distribuciones **asimétricas a la derecha (right-skewed)**, lo que sugiere la presencia de valores extremos elevados que distorsionan la media.

I.2.1 Calorías



Rango dinámico mostrado: ~18 a ~3517

Media: ~474

Mediana: ~313

Percentil 25: ~174

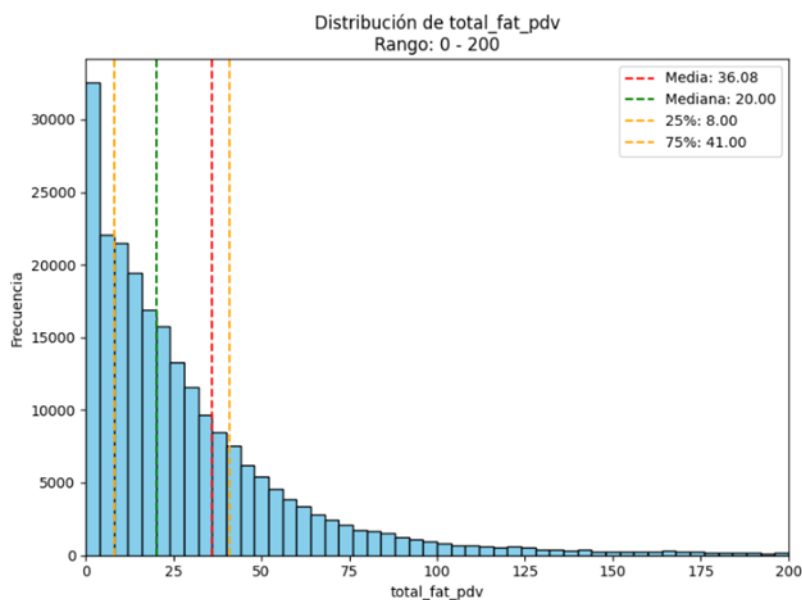
Percentil 75: ~520

Interpretación:

La media (474) es significativamente mayor que la mediana (313), lo cual indica una **distribución asimétrica hacia la derecha** (right-skewed).

Muchas recetas se concentran por debajo de ~500 calorías, pero existe un grupo relativamente pequeño con calorías muy elevadas, lo que empuja la media al alza.

I.2.2 Grasas Totales (%VD)



Rango dinámico mostrado: 0 a ~302

Media: ~36

Mediana: ~20

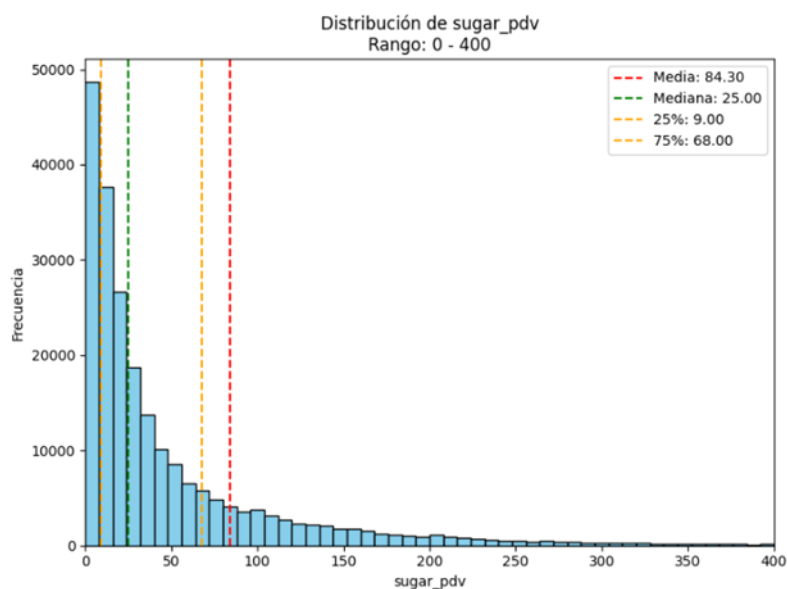
Percentil 25: ~8

Percentil 75: ~41

Interpretación:

También se observa una **asimetría derecha**: la mediana (20) está por debajo de la media (36). La mayoría de recetas presenta porcentajes de grasa relativamente moderados, pero algunas alcanzan valores muy altos que elevan la media.

I.2.3 Azúcares (%VD)



Rango dinámico mostrado: 0 a ~1142

Media: ~84

Mediana: ~25

Percentil 25: ~9

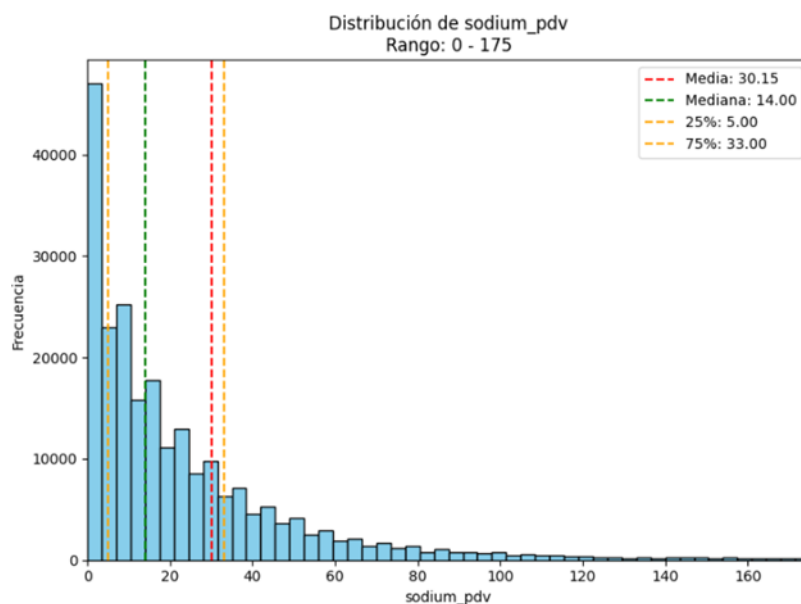
Percentil 75: ~68

Interpretación:

Existe una gran diferencia entre la media (84) y la mediana (25), lo cual denota un **grupo pequeño con valores de azúcar extremadamente altos**.

La mayoría de las recetas se sitúa por debajo de ~70-100, pero unos pocos valores por encima de 100 (o 200, 300, etc.) disparan la media.

I.2.4 Sodio (%VD)



Rango dinámico mostrado: 0 a ~219

Media: ~30

Mediana: ~14

Percentil 25: ~5

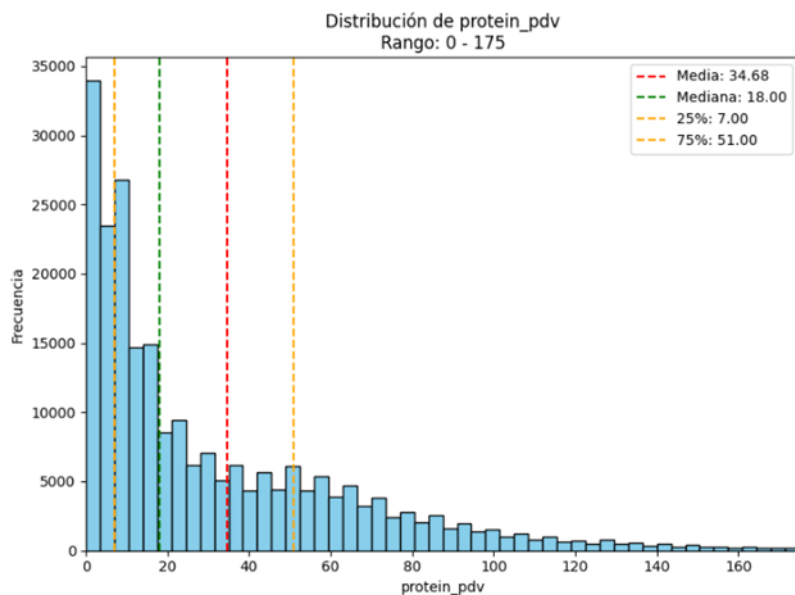
Percentil 75: ~33

Interpretación:

El valor medio (30) duplica la mediana (14), revelando otra **distribución sesgada** hacia valores altos.

Muchas recetas tienen un aporte de sodio moderado (por debajo de ~30), pero un subconjunto notable alcanza cifras más elevadas.

I.2.5 Proteínas (%VD)



Rango dinámico mostrado: 0 a ~188

Media: ~35

Mediana: ~18

Percentil 25: ~7

Percentil

75:

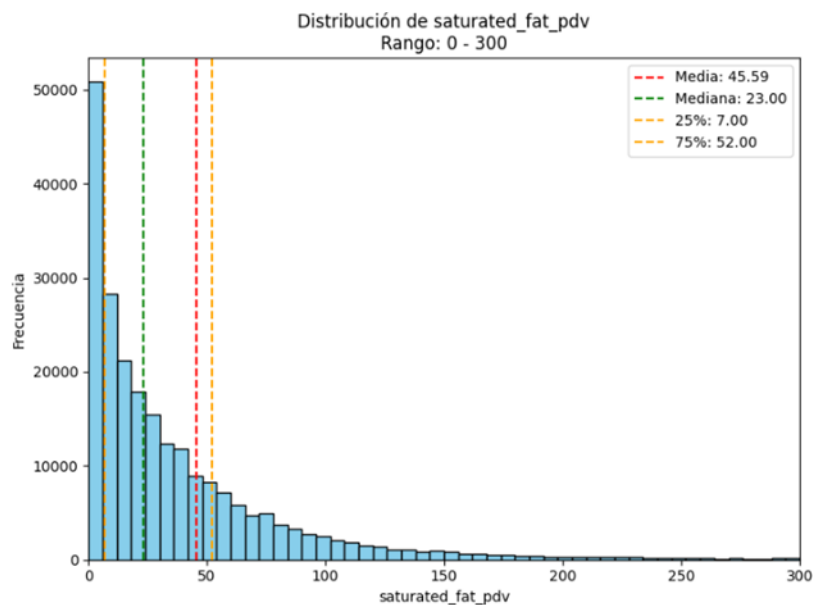
~51

Interpretación:

Con una mediana de 18 frente a una media de 35, se mantiene la **tendencia de valores extremos** que elevan la media.

La mayoría de recetas están por debajo de 50-60% VD en proteínas, pero hay un grupo pequeño muy elevado.

I.2.6 Grasas Saturadas (%VD)



Rango dinámico mostrado: 0 a ~404

Media: ~46

Mediana: ~23

Percentil 25: ~7

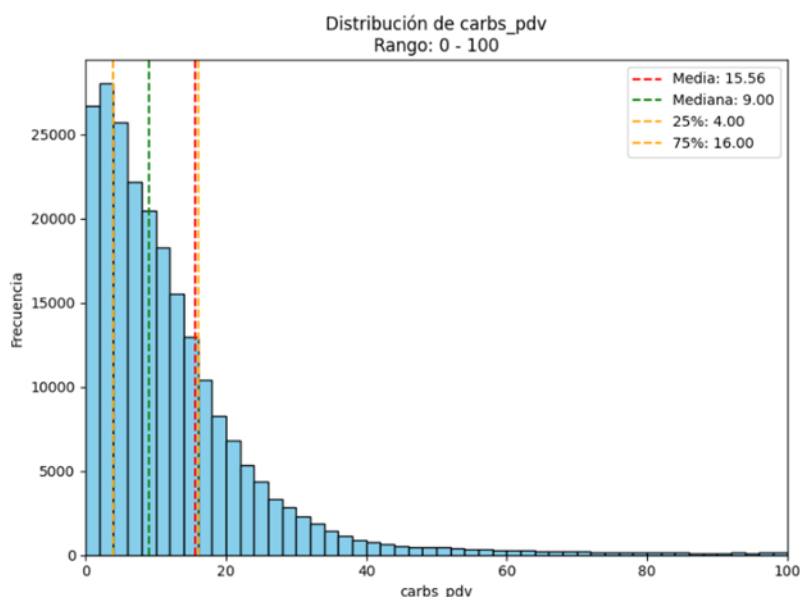
Percentil 75: ~52

Interpretación:

Se evidencia otra **distribución fuertemente asimétrica**: la media (46) es aproximadamente el doble de la mediana (23).

La mayoría de las recetas no superan ~50-60% VD de grasas saturadas, pero hay outliers muy altos que elevan la media.

I.2.7 Carbohidratos (%VD)



Rango dinámico mostrado: 0 a ~154

Media: ~16

Mediana: ~9

Percentil 25: ~4

Percentil 75: ~16

Interpretación:

Se mantiene la **asimetría**: la media (16) supera la mediana (9).

Aun así, la diferencia no es tan extrema como en azúcar o grasas saturadas, aunque sigue habiendo valores que llegan hasta ~100-150 y arrastran la media al alza.

I.3. Conclusiones generales del análisis

El estudio estadístico realizado sobre los valores nutricionales del conjunto de datos revela patrones consistentes en la distribución de los macronutrientes y micronutrientes analizados. En todos los casos, se observa una **distribución asimétrica positiva** (*right-skewed*), en la que la **media es superior a la mediana**, lo que indica la presencia de valores atípicos elevados (outliers) que distorsionan el promedio.

Esta asimetría sugiere que, aunque la mayoría de las recetas presentan aportes nutricionales moderados, existe un subconjunto reducido de recetas con valores extremadamente altos, especialmente en variables como **calorías, azúcar y grasas saturadas**. Este comportamiento afecta la representatividad de la media como medida

central, haciendo más apropiado el uso de estadísticas basadas en los **percentiles (P25 y P75)** y la **mediana** para caracterizar el perfil nutricional típico de una receta.

Además, el análisis ha permitido definir **criterios objetivos de clasificación nutricional**, segmentando cada variable en tres niveles —bajo, medio y alto— en función de los percentiles 25 y 75. Esta categorización no solo mejora la interpretabilidad de los datos, sino que también facilita la implementación de un sistema de recomendación más alineado con las necesidades específicas del usuario.

En resumen, el análisis descriptivo respalda la estrategia de clasificación empleada, proporciona una base sólida para el modelo de recomendación personalizado y pone de manifiesto la relevancia de utilizar métricas robustas frente a la presencia de valores extremos en los datos nutricionales.

Nota: Las variables nutricionales se expresan en **%VD (Porcentaje del Valor Diario)**, una medida estandarizada que indica qué porcentaje del requerimiento diario recomendado de un nutriente aporta una receta. Estos valores se basan en una dieta de referencia de 2.000 kcal diarias, y permiten comparar el impacto nutricional relativo entre diferentes recetas.

I.4. Clasificación nutricional basada en percentiles

Nutriente	Bajo (<P25)	Medio (P25–P75)	Alto (>P75)
Calorías	<174	174 – 520	>520
Grasas totales	<8	8 - 41	>41
Azúcar	<9	9 – 68	>68
Sodio	<5	5 – 33	>33
Proteínas	<7	7 - 51	>51
Grasas Saturadas	<7	7 – 52	>52
Carbohidratos	<4	4 - 16	>16

Anexo II – Resultados de Optimización de Hiperparámetros (HPO)

II.1 Configuración

- **Herramienta:** Optuna (TPE) integrada en pykeen.hpo_pipeline.
- **Modelos evaluados:** TransE, DistMult, RotatE.
- Espacio de búsqueda común
 - embedding_dim $\in \{64 \dots 512, \text{paso } 64\}$
 - lr $\in [5 \times 10^{-5}, 5 \times 10^{-3}]$ (log-scale)
 - batch_size $\in \{512, 1024, 2048\}$
 - Márgenes y pesos de regularización en rangos amplios.
- **Número de ensayos:** 30 por modelo (≈ 90 en total).
- **Dispositivo:** GPU local (RTX 4070).
- **Métrica objetivo:** hits_at_10, maximizar.
- **Duración total:** ≈ 4 h 23 min.

II.2 Rendimiento en validación

Modelo	Hits@19 max	Tiempo HPO
TransE	0,005	49 min
DistMult	0,4188	65 min
RotatE	0,4221	148 min

TransE supera a las restantes variantes con amplia ventaja y menor coste temporal, por lo que se elige para entrenar el sistema.

II.3 Mejores hiperparámetros hallados

La búsqueda identificó una configuración de TransE con 384 dimensiones de *embedding* y *scoring* L2 que maximiza la métrica objetivo. El margen óptimo para la función de pérdida de ranking se fijó en 1,82, mientras que el optimizador Adam alcanzó su mejor rendimiento con una tasa de aprendizaje de $3,2 \times 10^{-4}$. El entrenamiento se estabilizó con seis negativos por positivo y un lote de 512 ejemplos, convergiendo tras 200 épocas en la fase final.

El modelo resultante ocupa en disco unos 11 MB, lo que facilita su carga en frío (*cold start*) dentro de los límites de Cloud Functions y mantiene latencias de inferencia por debajo de los 60 ms.

II.4 Evaluación final (conjunto de prueba)

Con los hiperparámetros optimizados, TransE logró un hit@10 del 99,68 %, indicando que casi todas las recetas correctas aparecen entre las diez primeras sugerencias. El hits@3 asciende al 99,54 %, mientras que el hits@1 se sitúa en 28 %, revelando que, aunque el ranking es excelente a nivel de top-k, los empates semánticos inherentes al grafo dispersan la primera posición. El Mean Rank es 6,8 y la mediana de rango es 2, reforzando la rápida aparición de resultados relevantes. Estos números confirman la eficacia del modelo para filtrar decenas de miles de recetas con un coste computacional mínimo, si bien abren la puerta a futuras mejoras en la resolución de empates y en la afinación de preferencias nutricionales finas.

II.5 Conclusión

El proceso HPO identifica a TransE como la opción óptima, equilibrando precisión y ligereza. Sus latencias de inferencia (< 60 ms) y reducido tamaño posibilitan la ejecución *serverless* descrita en la arquitectura física (sección 4.4), asegurando un coste mínimo y una escalabilidad automática del servicio de recomendación.

Glosario de términos

Transfer Learning: técnica que reaprovecha los pesos de un modelo previamente entrenado para arrancar otro en una tarea diferente, reduciendo datos y tiempo de cómputo.

Fine-Tuning: fase de ajuste que actualiza parte (o la totalidad) de los pesos de un modelo pre-entrenado para especializarlo en un nuevo dominio.

Bounding Box: rectángulo (x, y, w, h) que delimita la posición y el tamaño de un objeto detectado por un modelo de visión como YOLO.

Embedding: vector numérico de dimensión fija que captura las características latentes de una entidad, palabra o nodo de grafo.

PyKEEN: biblioteca Python que implementa numerosos algoritmos de KGE, con utilidades para entrenamiento, evaluación y búsqueda de hiperparámetros.

Optuna: framework de optimización bayesiana que automatiza la búsqueda de hiperparámetros.

Early Stopping: criterio que detiene el entrenamiento cuando la métrica de validación deja de mejorar, evitando sobreajuste y ahorrando cómputo.

Microservicio: unidad de despliegue pequeña y autónoma que encapsula una única funcionalidad.

Serverless: modelo en la nube en el que el proveedor crea, escala y apaga automáticamente la infraestructura, el desarrollador despliega funciones/contenedores y paga solo por la ejecución efectiva, sin gestionar servidores ni reservar capacidad.

Cold Start: intervalo inicial en el que un servicio serverless lanza una nueva instancia para atender la primera petición; implica cargar dependencias y modelos en memoria, provocando una latencia mayor que en invocaciones siguientes “en caliente”.

Endpoint: URL o punto de entrada expuesto por un servicio que recibe peticiones y devuelve respuestas.

Hosting: provisión y publicación de un servicio, modelo o archivo en una infraestructura accesible.

Lazy Loading: patrón que retrasa la carga de un recurso pesado (modelo, datos) hasta la primera vez que realmente se necesita, reduciendo el cold-start.

Serverless: modelo de nube donde el proveedor abstrae la gestión de servidores; el usuario paga solo por el tiempo de ejecución de su código.

Cloud Function (Gen 2): función serverless de Google Cloud que se activa por eventos o llamadas HTTP y escala de cero a N instancias.

Pub/Sub: servicio de mensajería asíncrona publish–subscribe de Google Cloud que desacopla productores (publishers) y consumidores (subscribers).

Topic (Pub/Sub): canal lógico al que los productores publican mensajes y del que las suscripciones extraen esos mensajes para procesarlos.

Firestore: base de datos NoSQL de documentos de Google Cloud con lecturas/escrituras de baja latencia y facturación por operación.

Bucket (Cloud Storage): contenedor de objetos binarios (modelos, CSV, imágenes) con control de acceso granular.

